



US 20250279109A1

(19) **United States**

(12) **Patent Application Publication**
Heitkaemper et al.

(10) **Pub. No.: US 2025/0279109 A1**

(43) **Pub. Date: Sep. 4, 2025**

(54) **STREAMING, ARRAY-AGNOSTIC, FULL-
AND SUB-BAND MODELING FRONT- END
FOR ROBUST AUTOMATIC SPEECH
RECOGNITION**

(52) **U.S. Cl.**
CPC **G10L 21/0208** (2013.01); **G10L 15/16**
(2013.01); **G10L 2021/02082** (2013.01); **G10L**
2021/02087 (2013.01)

(71) Applicant: **Google LLC**, Mountain View, CA (US)

(72) Inventors: **Jens Heitkaemper**, Mountain View, CA
(US); **Joseph Peter Caroselli, JR.**, Palo
Alto, CA (US); **Arun Narayanan**,
Milpitas, CA (US); **Nathan David**
Howard, Mountain View, CA (US)

(73) Assignee: **Google LLC**, Mountain View, CA (US)

(21) Appl. No.: **19/057,217**

(22) Filed: **Feb. 19, 2025**

Related U.S. Application Data

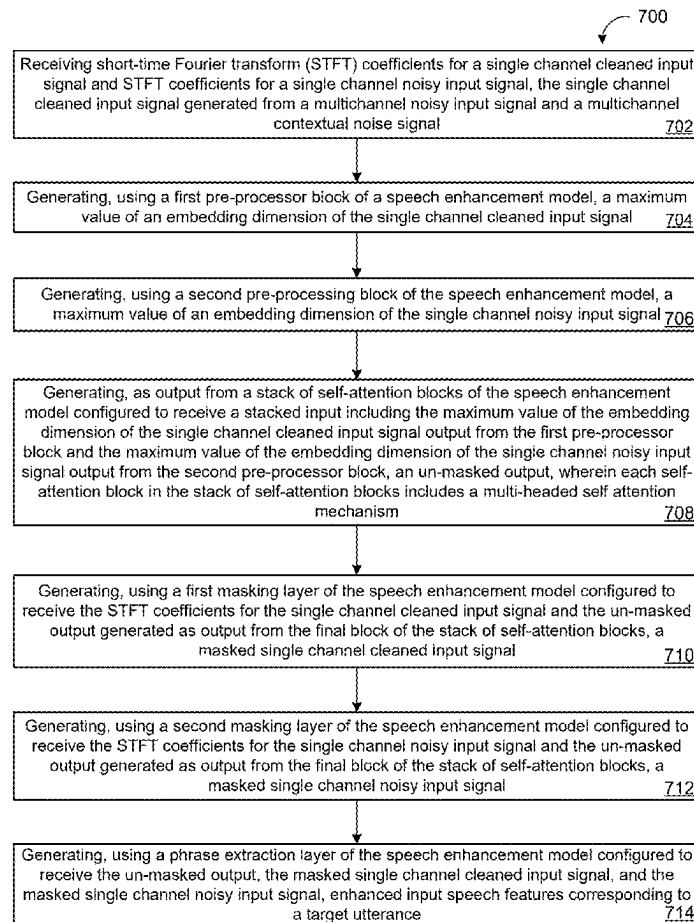
(60) Provisional application No. 63/560,063, filed on Mar.
1, 2024.

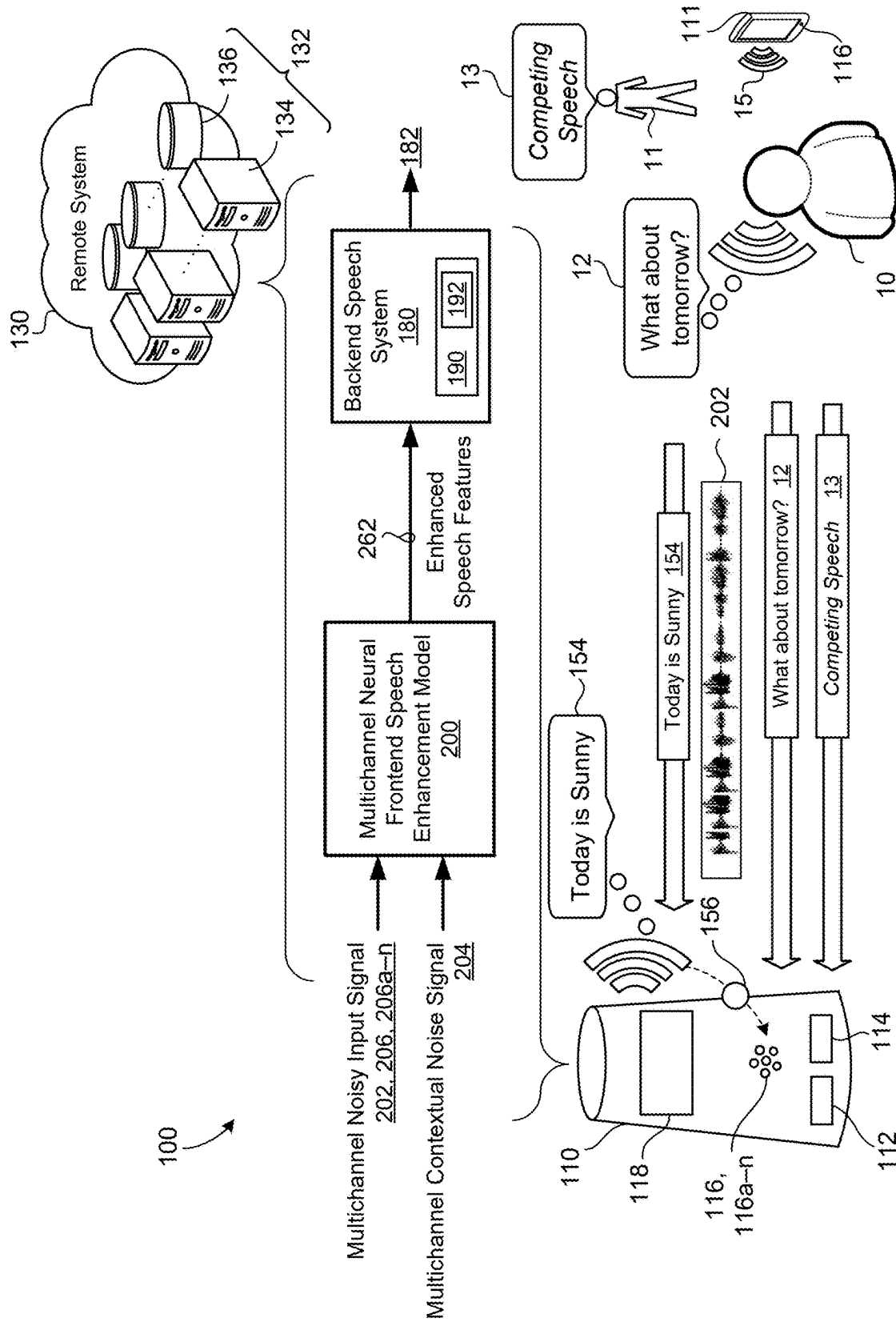
Publication Classification

(51) **Int. Cl.**
G10L 21/0208 (2013.01)
G10L 15/16 (2006.01)

(57) **ABSTRACT**

A multichannel neural frontend speech enhancement model includes a first pre-processor block, a second pre-processor block, a stack of self-attention blocks, a first masking layer, a second masking layer, and a phrase extraction layer. The first pre-processor receives short-time Fourier transform (STFT) coefficients for a single channel cleaned input signal and generates a maximum value of an embedding dimension of the cleaned input signal. The second pre-processor receives STFT coefficients for a single channel noisy input signal and generates a maximum value of an embedding dimension of the noisy input signal. The stack of self-attention blocks receives a stacked input of the embedding dimensions of the cleaned input signal and the noisy input signal and generates an un-masked output. The phrase extraction layer receives the un-masked output, a masked cleaned input signal, and a masked noisy input signal, and generate enhanced input speech features.





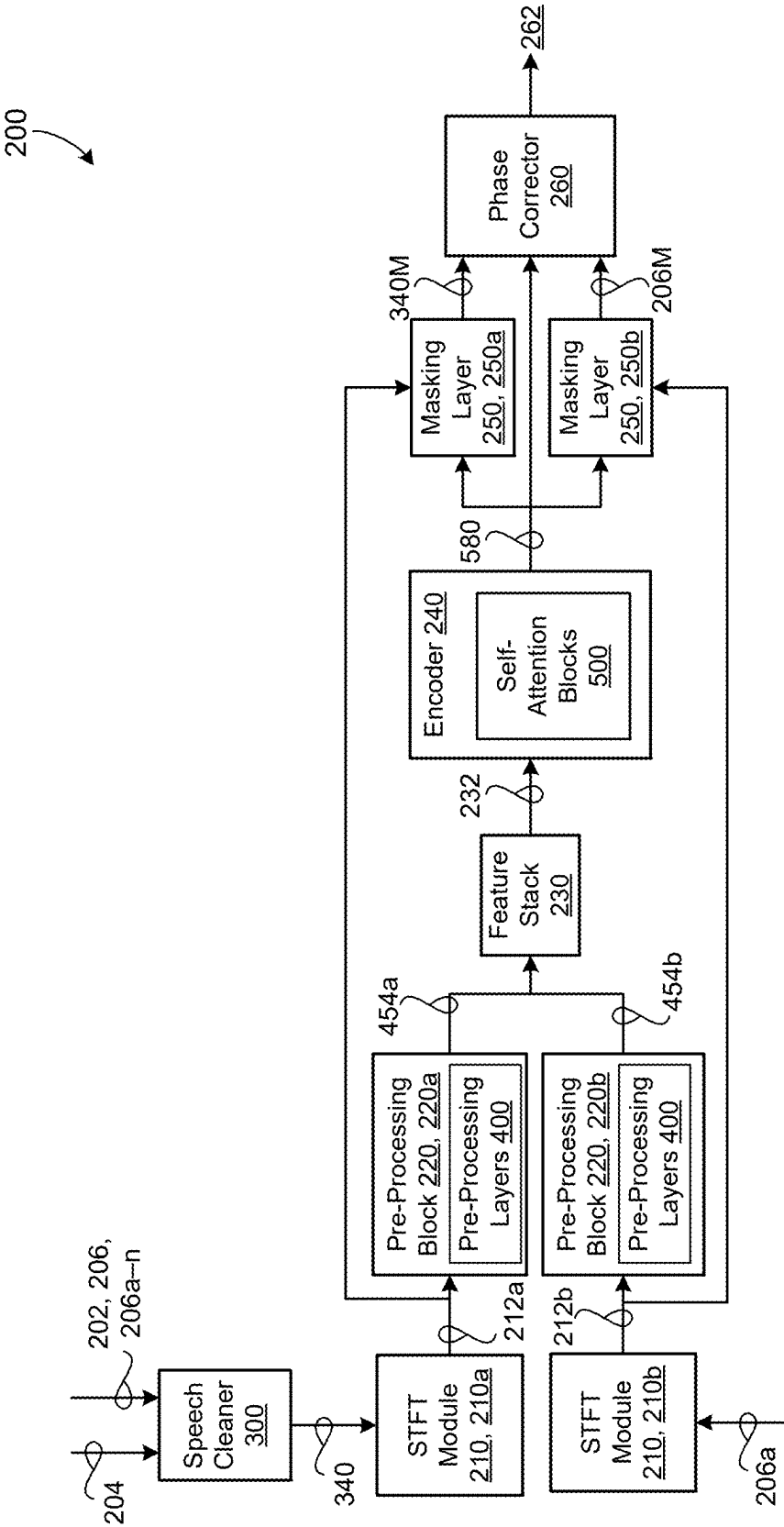


FIG. 2

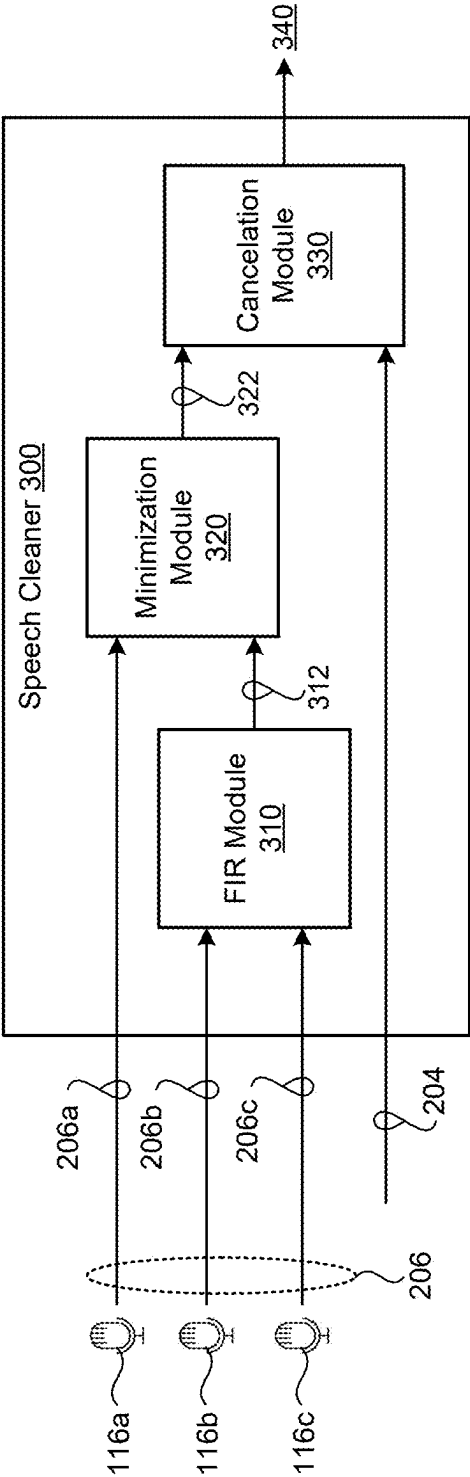


FIG. 3

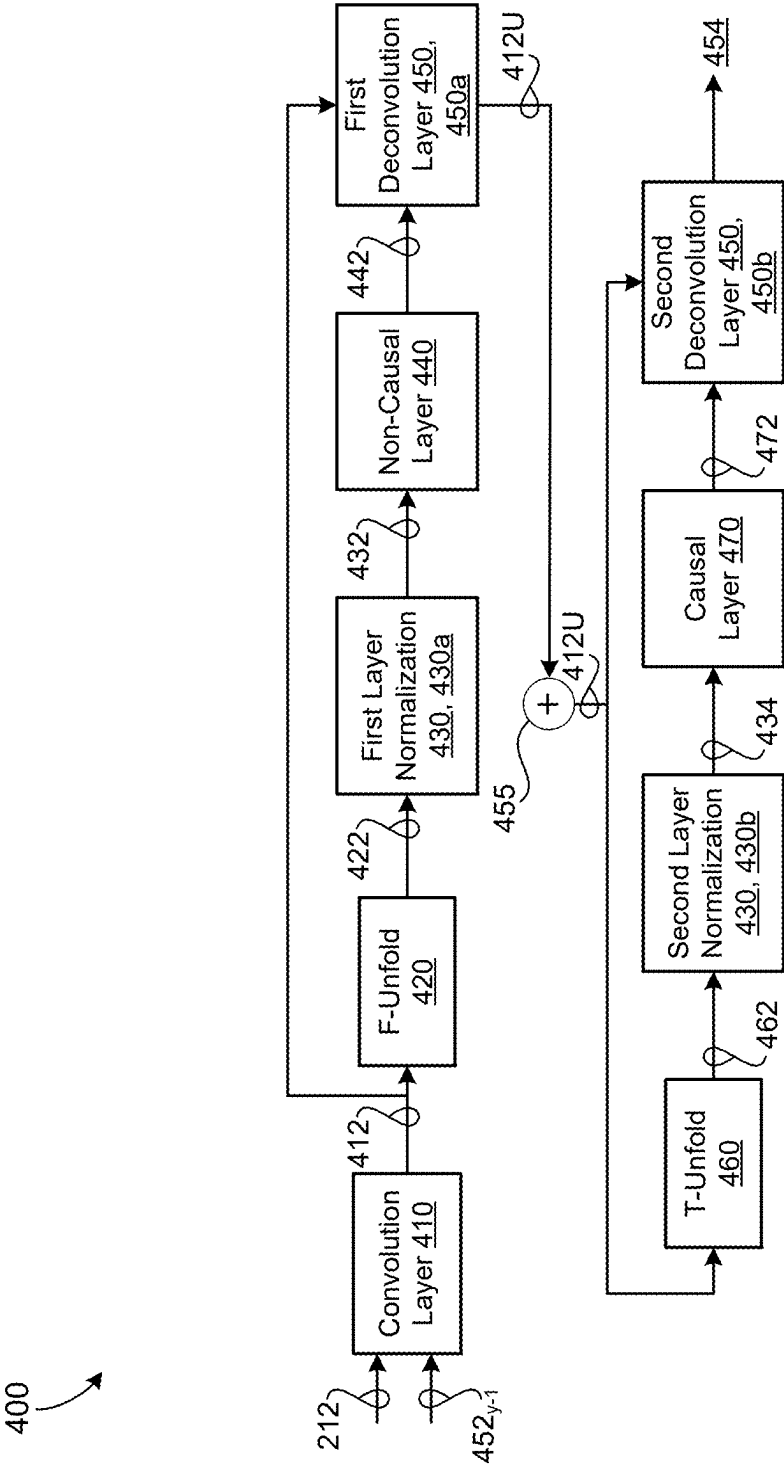


FIG. 4

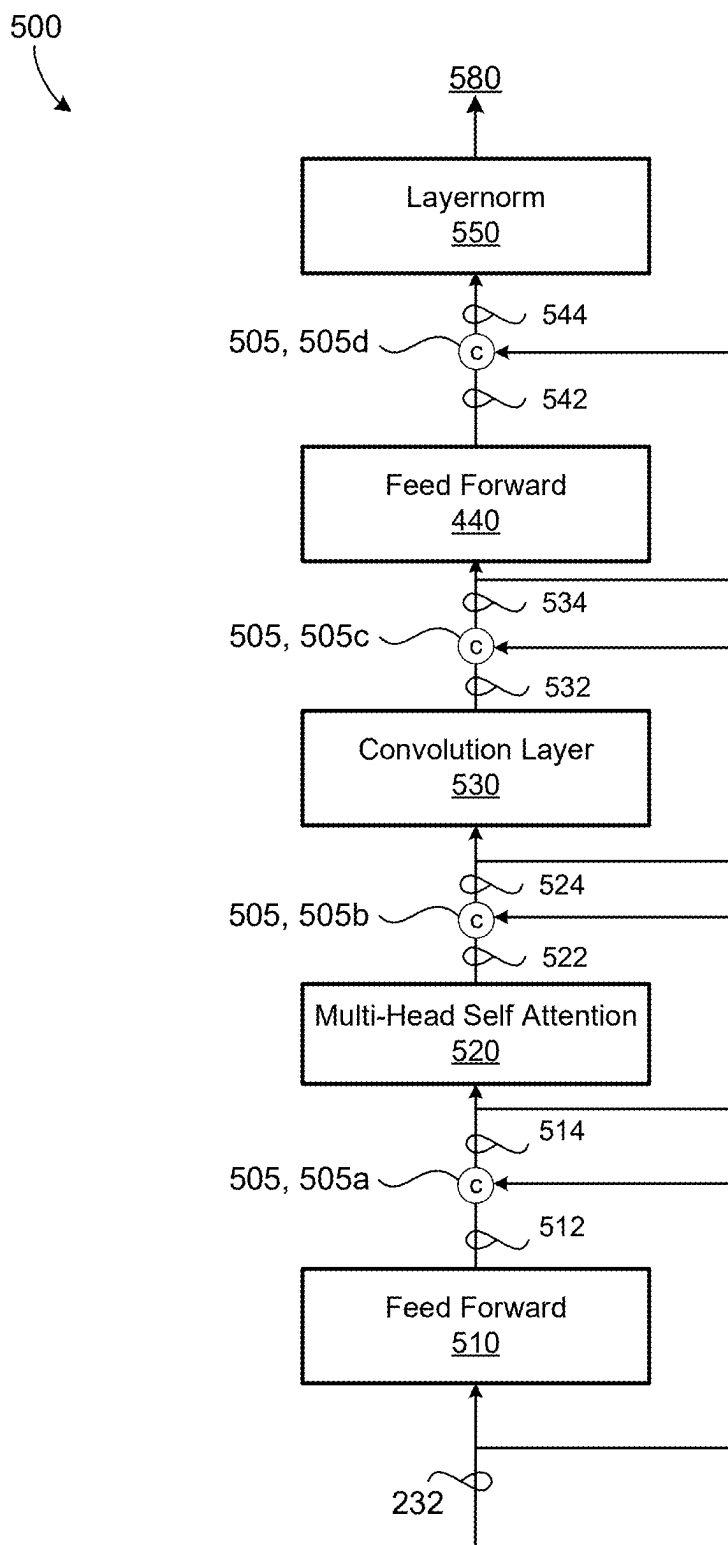


FIG. 5

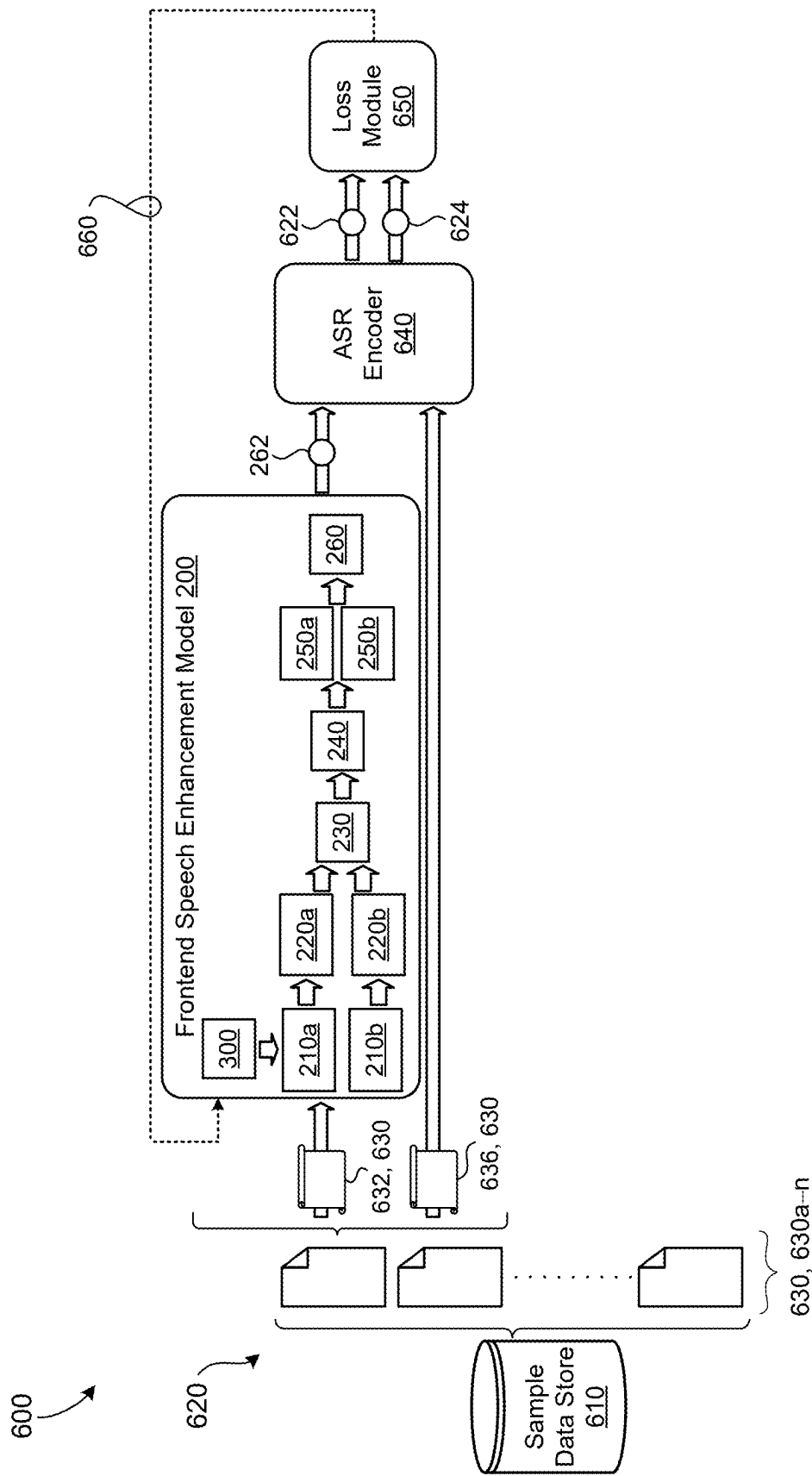


FIG. 6

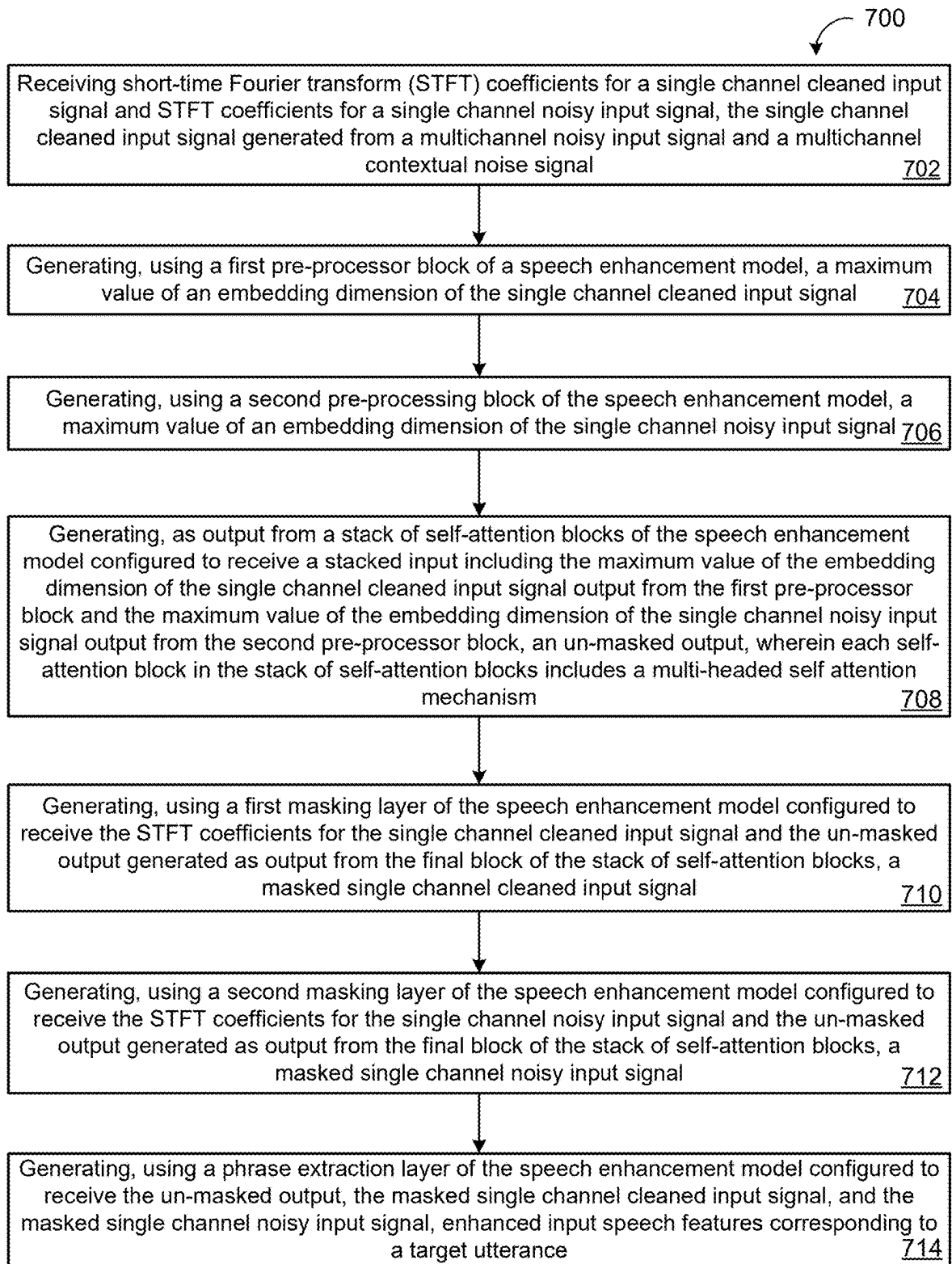


FIG. 7

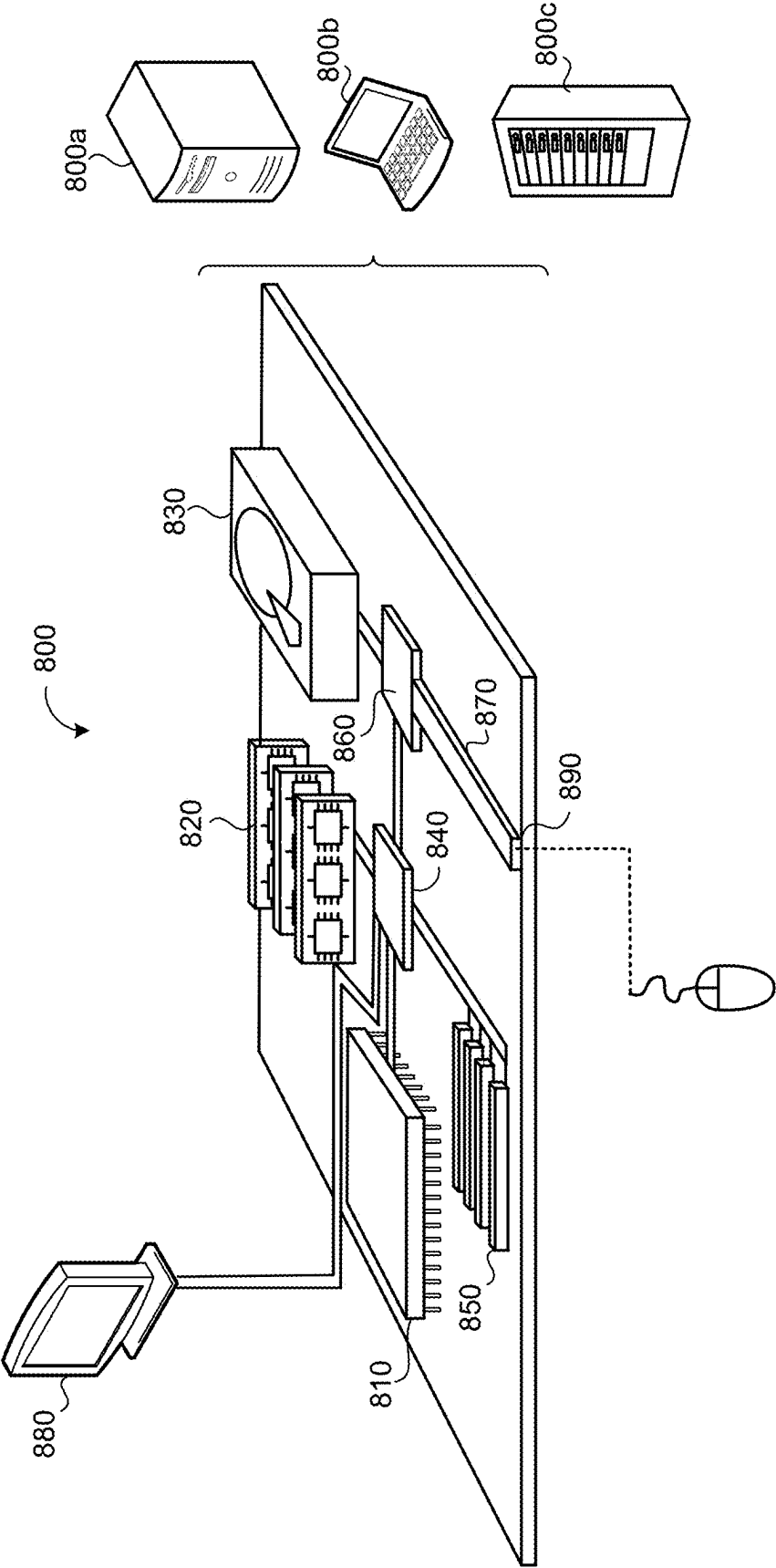


FIG. 8

STREAMING, ARRAY-AGNOSTIC, FULL- AND SUB-BAND MODELING FRONT- END FOR ROBUST AUTOMATIC SPEECH RECOGNITION

CROSS REFERENCE TO RELATED APPLICATIONS

[0001] This U.S. patent application claims priority under 35 U.S.C. § 119 (e) to U.S. Provisional Application 63/560, 063, filed on Mar. 1, 2024. The disclosure of this prior application is considered part of the disclosure of this application and is hereby incorporated by reference in its entirety.

TECHNICAL FIELD

[0002] This disclosure relates to a streaming, array-agnostic, full- and sub-band modeling front-end for robust automatic speech recognition (ASR).

BACKGROUND

[0003] Robustness of automatic speech recognition (ASR) systems has significantly improved over the years with the advent of neural network-based end-to-end models, large-scale training data, and improved strategies for augmenting training data. Nevertheless, various conditions such as reverberation, significant background noise, and competing speech significantly deteriorate performance of ASR systems. A joint ASR model may be trained to handle these conditions. However, isolating speech in background conditions including speech-based noise and non-speech based noise is particularly challenging.

SUMMARY

[0004] One aspect of the present disclosure provides a multichannel neural frontend speech enhancement model for speech recognition that includes a first pre-processor block, a second pre-processor block, a stack of self-attention blocks each having a multi-headed self attention mechanism, a first masking layer, a second masking layer, and a phase extraction layer. The first pre-processor receives, as input, short-time Fourier transform (STFT) coefficients for a single channel cleaned input signal, the single channel cleaned input signal generated from a multichannel noisy input signal and a multichannel contextual noise signal, and generates, as output, a maximum value of an embedding dimension of the single channel cleaned input signal. The second pre-processor block receives, as input, STFT coefficients for a single channel noisy input signal, and generates, as output, a maximum value of an embedding dimension of the single channel noisy input signal. The stack of self-attention blocks receives, as input, at an initial block of the stack of self-attention blocks, a stacked input including the maximum value of the embedding dimension of the single channel cleaned input signal output from the first pre-processor block and the maximum value of the embedding dimension of the single channel noisy input signal output from the second pre-processor block, and generates, as output from a final block of the stack of self-attention blocks, an un-masked output. The first masking layer receives, as input, the STFT coefficients for the single channel cleaned input signal and the un-masked output generated as output from the final block of the stack of self-attention blocks, and generates, as output, a masked

single channel cleaned input signal. The second masking layer receives, as input, the STFT coefficients for the single channel noisy input signal and the un-masked output generated as output from the final block of the stack of self-attention blocks, and generates, as output, a masked single channel noisy input signal. The phrase extraction layer receives, as input, the un-masked output, the masked single channel cleaned input signal, and the masked single channel noisy input signal, and generates, as output, enhanced speech features corresponding to a target utterance.

[0005] Implementations of the disclosure may include one or more of the following optional features. In some implementations, the first pre-processor block includes a plurality of pre-processing layers, each pre-processing layer including a convolutional layer, an unfold frequency operation, a first layer normalization, a non-causal context-aware layer, a first deconvolutional layer, an unfold time operation, a second layer normalization, a causal context-aware layer, and a second deconvolutional layer. The convolutional layer is configured to receive the maximum value of the embedding dimension of each corresponding frame of the single channel cleaned input signal output by a preceding pre-processing layer and calculate a corresponding embedding vector of size E for each of multiple time-frequency bins. The unfold frequency operation is configured to unfold the corresponding embedding vector over a frequency dimension, the first layer normalization is configured to normalize each embedding vector, and the non-causal context-aware layer is configured to correlate the frequency dimension as a context dimension. The first deconvolutional layer is configured to process and combine an output of the non-causal context-aware layer with the corresponding embedding vectors calculated by the convolutional layer to provide updated embedding vectors to a residual connection. The unfold time operation is configured to unfold the updated embedding vectors over a time dimension, the second layer normalization is configured to normalize each of the updated embedding vectors unfolded over the time dimension, and the causal context-aware layer is configured to correlate the time dimension as the context dimension. The second deconvolutional layer is configured to process and combine an output of the causal context-aware layer with the updated embedding vectors provided to the residual connection to output the maximum value of the embedding dimension of the single channel cleaned input signal. In these implementations, the convolutional layer may include a causal two-dimensional convolutional layer with a 3x3 kernel and a stride equal to one.

[0006] In some examples, the second pre-processor block includes a plurality of pre-processing layers, each pre-processing layer including a convolutional layer an unfold frequency operation, a first layer normalization, a non-causal context-aware layer, a first deconvolutional layer, an unfold time operation, a second layer normalization, a causal context-aware layer, and a second deconvolutional layer. The convolutional layer is configured to receive the maximum value of the embedding dimension of each corresponding frame of the single channel noisy input signal output by a preceding pre-processing layer and calculate a corresponding embedding vector of size E for each of multiple time-frequency bins. The unfold frequency operation is configured to unfold the corresponding embedding vector over a frequency dimension, the first layer normalization is configured to normalize each embedding vector, and the non-

causal context-aware layer is configured to correlate the frequency dimension as a context dimension. The first deconvolutional layer is configured to process and combine an output of the non-causal context-aware layer with the corresponding embedding vectors calculated by the convolutional layer to provide updated embedding vectors to a residual connection. The unfold time operation is configured to unfold the updated embedding vectors over a time dimension, the second layer normalization is configured to normalize each of the updated embedding vectors unfolded over the time dimension, and the causal context-aware layer is configured to correlate the time dimension as the context dimension. The second deconvolutional layer is configured to process and combine an output of the causal context-aware layer with the updated embedding vectors provided to the residual connection to output the maximum value of the embedding dimension of the single channel noisy output signal. In these implementations, the convolutional layer may include a causal two-dimensional convolutional layer with a 3×3 kernel and a stride equal to one.

[0007] In some implementations, the stack of self-attention blocks includes a stack of Conformer blocks. In these implementations, the stack of Conformer blocks may include four Conformer blocks. In some examples, the speech enhancement model executes on data processing hardware residing on a user device. Here, the user device is configured to capture the target utterance and the multichannel contextual noise signal via an array of microphones of the user device. In these examples, the speech enhancement model may be agnostic to a number of microphones in the array of microphones.

[0008] In some implementations, the speech enhancement model further includes a speech cleaner executing an adaptive noise cancellation algorithm that generates the single channel cleaned input signal by applying a finite impulse response (FIR) filter on all channels of the multichannel noisy input signal except for a first channel of the multichannel noisy input signal to generate a summed output, and subtracting the summed output from the first channel of the multichannel noisy input signal. In some examples, a backend speech system is configured to process the enhanced input speech features corresponding to the target utterance. In these examples, the backend speech system includes at least one of an automatic speech recognition (ASR) model or an audio or audio-video calling application.

[0009] In some implementations, the speech enhancement model is trained jointly with a backend automatic speech recognition (ASR) model using a spectral loss and an ASR loss. In these implementations, the spectral loss may be based on an L1 loss function and L2 loss function distance between an estimated ratio mask and an ideal ratio mask. Here, the ideal ratio mask is computed using reverberant speech and reverberant noise. Additionally or alternatively, the ASR loss is computed by generating, using an ASR encoder of the ASR model configured to receive enhanced speech features predicted by the speech enhancement model for a training utterance as input, predicted outputs of the ASR encoder for the enhanced speech features, generating, using the ASR encoder configured to receive target speech features for the training utterance as input, target outputs of the ASR encoder for the target speech features. Here, computing the ASR loss is based on the predicted outputs of

the ASR encoder for the enhanced speech features and the target outputs of the ASR encoder for the target speech features.

[0010] Another aspect of the disclosure provides a computer-implemented method that when executed on data processing hardware causes the data processing hardware to perform operations that include receiving short-time Fourier transform (STFT) coefficients for a single channel cleaned input signal and STFT coefficients for a single channel noisy input signal, the single channel cleaned input signal generated from a multichannel noisy input signal and a multichannel contextual noise signal. The operations also include generating, using a first pre-processor block of a speech enhancement model, a maximum value of an embedding dimension of the single channel cleaned input signal, and generating using a second pre-processing block of the speech enhancement model, a maximum value of an embedding dimension of the single channel noisy input signal. The operations further include generating, as output from a stack of self-attention blocks of the speech enhancement model configured to receive a stacked input including the maximum value of the embedding dimension of the single channel cleaned input signal output from the first pre-processor block and the maximum value of the embedding dimension of the single channel noisy input signal output from the second pre-processor block, an un-masked output. Here, each self-attention block in the stack of self-attention blocks includes a multi-headed self attention mechanism. The operations also include generating, using a first masking layer of the speech enhancement model configured to receive the STFT coefficients for the single channel cleaned input signal and the un-masked output generated as output from the final block of the stack of self-attention blocks, a masked single channel cleaned input signal, and generating, using a second masking layer of the speech enhancement model configured to receive the STFT coefficients for the single channel noisy input signal and the un-masked output generated as output from the final block of the stack of self-attention blocks, a masked single channel noisy input signal. The operations further include generating, using a phrase extraction layer of the speech enhancement model configured to receive the un-masked output, the masked single channel cleaned input signal, and the masked single channel noisy input signal, enhanced input speech features corresponding to a target utterance. This aspect may include one or more of the following optional features. In some implementations, the first pre-processor block includes a plurality of pre-processing layers, each pre-processing layer including a convolutional layer an unfold frequency operation, a first layer normalization, a non-causal context-aware layer, a first deconvolutional layer, an unfold time operation, a second layer normalization, a causal context-aware layer, and a second deconvolutional layer. The convolutional layer is configured to receive the maximum value of the embedding dimension of each corresponding frame of the single channel cleaned input signal output by a preceding pre-processing layer and calculate a corresponding embedding vector of size E for each of multiple time-frequency bins. The unfold frequency operation is configured to unfold the corresponding embedding vector over a frequency dimension, the first layer normalization is configured to normalize each embedding vector, and the non-causal context-aware layer is configured to correlate the frequency dimension as a context dimension. The first deconvolutional layer is

configured to process and combine an output of the non-causal context-aware layer with the corresponding embedding vectors calculated by the convolutional layer to provide updated embedding vectors to a residual connection. The unfold time operation is configured to unfold the updated embedding vectors over a time dimension, the second layer normalization is configured to normalize each of the updated embedding vectors unfolded over the time dimension, and the causal context-aware layer is configured to correlate the time dimension as the context dimension. The second deconvolutional layer is configured to process and combine an output of the causal context-aware layer with the updated embedding vectors provided to the residual connection to output the maximum value of the embedding dimension of the single channel cleaned input signal. In these implementations, the convolutional layer may include a causal two-dimensional convolutional layer with a 3×3 kernel and a stride equal to one.

[0011] In some examples, the second pre-processor block includes a plurality of pre-processing layers, each pre-processing layer including a convolutional layer an unfold frequency operation, a first layer normalization, a non-causal context-aware layer, a first deconvolutional layer, an unfold time operation, a second layer normalization, a causal context-aware layer, and a second deconvolutional layer. The convolutional layer is configured to receive the maximum value of the embedding dimension of each corresponding frame of the single channel noisy input signal output by a preceding pre-processing layer and calculate a corresponding embedding vector of size E for each of multiple time-frequency bins. The unfold frequency operation is configured to unfold the corresponding embedding vector over a frequency dimension, the first layer normalization is configured to normalize each embedding vector, and the non-causal context-aware layer is configured to correlate the frequency dimension as a context dimension. The first deconvolutional layer is configured to process and combine an output of the non-causal context-aware layer with the corresponding embedding vectors calculated by the convolutional layer to provide updated embedding vectors to a residual connection. The unfold time operation is configured to unfold the updated embedding vectors over a time dimension, the second layer normalization is configured to normalize each of the updated embedding vectors unfolded over the time dimension, and the causal context-aware layer is configured to correlate the time dimension as the context dimension. The second deconvolutional layer is configured to process and combine an output of the causal context-aware layer with the updated embedding vectors provided to the residual connection to output the maximum value of the embedding dimension of the single channel noisy output signal. In these implementations, the convolutional layer may include a causal two-dimensional convolutional layer with a 3×3 kernel and a stride equal to one.

[0012] In some implementations, the stack of self-attention blocks includes a stack of Conformer blocks. In these implementations, the stack of Conformer blocks may include four Conformer blocks. In some examples, the speech cleaner, the stack of self-attention blocks, and the masking layer execute on the data processing hardware residing on a user device. Here, the user device is configured to capture the target utterance and the multichannel contextual noise signal via an array of microphones of the user

device. In these examples, the speech enhancement model may be agnostic to a number of microphones in the array of microphones.

[0013] In some implementations, the speech enhancement model further includes a speech cleaner executing an adaptive noise cancelation algorithm that generates the single channel cleaned input signal by applying a finite impulse response (FIR) filter on all channels of the multichannel noisy input signal except for a first channel of the multichannel noisy input signal to generate a summed output, and subtracting the summed output from the first channel of the multichannel noisy input signal. In some examples, a backend speech system is configured to process the enhanced input speech features corresponding to the target utterance. In these examples, the backend speech system includes at least one of an automatic speech recognition (ASR) model or an audio or audio-video calling application.

[0014] In some implementations, the speech enhancement model is trained jointly with a backend automatic speech recognition (ASR) model using a spectral loss and an ASR loss. In these implementations, the spectral loss may be based on an L1 loss function and L2 loss function distance between an estimated ratio mask and an ideal ratio mask. Here, the ideal ratio mask is computed using reverberant speech and reverberant noise. Additionally or alternatively, the ASR loss is computed by generating, using an ASR encoder of the ASR model configured to receive enhanced speech features predicted by the speech enhancement model for a training utterance as input, predicted outputs of the ASR encoder for the enhanced speech features, generating, using the ASR encoder configured to receive target speech features for the training utterance as input, target outputs of the ASR encoder for the target speech features. Here, computing the ASR loss is based on the predicted outputs of the ASR encoder for the enhanced speech features and the target outputs of the ASR encoder for the target speech features.

[0015] The details of one or more implementations of the disclosure are set forth in the accompanying drawings and the description below. Other aspects, features, and advantages will be apparent from the description and drawings, and from the claims.

DESCRIPTION OF DRAWINGS

[0016] FIG. 1 is a schematic view of a system that includes a user communicating a spoken target utterance to a speech-enabled user device.

[0017] FIG. 2 is a schematic view of a multichannel neural frontend speech enhancement model of FIG. 1.

[0018] FIG. 3 is a schematic view of a speech cleaner of the multichannel neural frontend speech enhancement model.

[0019] FIG. 4 is a schematic view of a pre-processing block of the multichannel neural frontend speech enhancement model.

[0020] FIG. 5 is a schematic view of a self-attention conformer block of the multichannel neural frontend speech enhancement model.

[0021] FIG. 6 is a schematic view of an example training process for jointly training a contextual frontend processing model and an automatic speech recognition model.

[0022] FIG. 7 is an example flowchart of an example arrangement of operations for a method of automatic speech recognition using a multichannel neural frontend speech enhancement model.

[0023] FIG. 8 is a schematic view of an example computing device that may be used to implement the systems and methods described herein.

[0024] Like reference symbols in the various drawings indicate like elements.

DETAILED DESCRIPTION

[0025] Robustness of automatic speech recognition (ASR) systems has significantly improved over the years with the advent of neural network-based end-to-end models, large-scale training data, and improved strategies for augmenting training data. Nevertheless, background interference can significantly deteriorate the ability of ASR systems to accurately recognize speech directed toward the ASR system. Moreover, users increasingly expect fast response times when interacting with ASR systems. Background interference can be broadly classified into three groups: device echo; background noise; and competing speech. While separate ASR models may be trained to handle each of these background interference groups in isolation, the difficulty in maintaining multiple task/condition-specific ASR models and switching between the models on the fly during use is not practical.

[0026] Device echo may correspond to playback audio output from devices, such as smart home speakers, whereby the playback audio is recorded as echo and can affect performance of a backend speech system, such as an ASR system. Particularly, degradation of performance of the backend speech system is especially severe if the playback audio contains audible speech, e.g., a text-to-speech (TTS) response from a digital assistant.

[0027] Background noise with non-speech characteristics is usually well-handled using data augmentation strategies like multi-style training (MTR) of the ASR models. Here, a room simulator is used to add noise to the training data, which is then carefully weighted with clean data during training to get a good balance in performance between clean and noisy conditions. As a result, large scale ASR models are robust to moderate levels of non-speech noise. However, background noise can still affect performance of backend speech systems in the presence of low signal-to-noise ratio (SNR) conditions.

[0028] Unlike non-speech background noise, competing speech is quite challenging for ASR models that are trained to recognize a single speaker. Training ASR models with multi-talker speech can pose problems in itself, since it is hard to disambiguate which speaker to focus on during inference. Using models that recognize multiple speakers is also sub-optimal since it is hard to know ahead of time how many users to support. Furthermore, such multi-speaker models typically have degraded performance in single-speaker settings, which is undesirable.

[0029] The three aforementioned classes of background interference have typically been addressed in isolation of one another, each using separate modeling strategies. Speech separation has received significant attention in the recent literature using techniques like deep clustering, permutation invariant training, and speaker embeddings. When using speaker embeddings, the target speaker of interest is assumed to be known a priori. Techniques developed for

speaker separation have also been applied to remove non-speech noise, with modifications to the training data. Acoustic Echo Cancellation (AEC) has also been studied in isolation or together in the presence of background noise. It is well known that improving speech quality does not always improve ASR performance since the distortions introduced by non-linear processing can adversely affect ASR performance. One way to mitigate discrepancies between an enhancement frontend initially processing incoming audio and the resulting ASR performance is to jointly train the enhancement frontend together with the backend ASR model.

[0030] Moreover, as the application of large-scale multi-domain and multi-lingual ASR models continues to gain interest, the training data for these ASR models typically covers various acoustic and linguistic use cases (e.g., voice search and video captioning), thereby making it challenging to simultaneously address harsher noise conditions. As a result, it is often convenient to train and maintain separate frontend feature processing models capable of handling adverse conditions, without combining it with the backend ASR model.

[0031] Implementations herein are directed toward training a frontend speech enhancement model for improving robustness of ASR. The model is practical from the standpoint that it is difficult, if not impossible, to know what class of background interference to address ahead of time, particularly in a streaming ASR setting. Specifically, the frontend speech enhancement model includes a contextual enhancement neural network (CENN) capable of making use of a multichannel noisy input signal and a multichannel contextual noise signal. For speech enhancement and separation, the noise context, i.e., a few seconds of audio before the target utterance to be recognized, carries useful information about the acoustic context. The CENN employs a respective neural network architecture configured to ingest the noisy input and the contextual input to produce enhanced input speech features that may be passed to a backend speech system, such as, an ASR model that may process the enhanced input speech features to generate a speech recognition result for the target utterance. Notably, though the frontend speech enhancement model is designed to operate with a multi-channel array, the frontend speech enhancement model itself is agnostic as to the number of channels in the array or their configuration.

[0032] Referring to FIG. 1, in some implementations, a system 100 includes a user 10 communicating a spoken target utterance 12 to a speech-enabled user device 110 (also referred to as a device 110 or a user device 110) in a speech environment. The user 10 (i.e., speaker of the utterance 12) may speak the target utterance 12 as a query or a command to solicit a response from the device 110. The device 110 is configured to capture sounds from one or more users 10, 11 within the speech environment. Here, the audio sounds may refer to a spoken utterance 12 by the user 10 that functions as an audible query, a command for the device 110, or an audible communication captured by the device 110. Speech-enabled systems of the device 110 or associated with the device 110 may field the query for the command by answering the query and/or causing the command to be performed.

[0033] Various types of background interference may interfere with the ability of a backend speech system 180 to process the target utterance 12 that specifies the query or command for the device 110. As aforementioned, the back-

ground interference may include one or more of a device echo corresponding to playback audio 154 output from the user device (e.g., a smart speaker) 110, competing speech 13 such as utterances other than the target utterance 12 spoken by one or more other users 11 that are not directed toward the device 110, and background noise with non-speech characteristics such as a ringtone 15 from a separate user device 111. Implementations herein employ a multichannel neural frontend speech enhancement model 200 (also referred to as a model 200 or a frontend speech enhancement model 200) that executes on the device 110 and is configured to receive, as input, a multichannel noisy input signal 202 including speech features corresponding to the target utterance 12 and the background interference, and a multichannel contextual noise signal 204 and generate, as output, enhanced input speech features 262 corresponding to the target utterance 12 by processing the multichannel noisy input signal 202 and the multichannel contextual noise signal 204 to remove the background interference. The multichannel noisy input signal 202 includes one or more channels 206, 206a-n of audio. A backend speech system 180 may then process the enhanced input speech features 262 to generate an output 182. Notably, the multichannel neural frontend speech enhancement model 200 effectively removes (i.e., masks) the presence of background interference recorded by the device 110 when the user 10 spoke the target utterance 12 such that the enhanced input speech features 262 provided to the backend speech system 180 convey the speech (i.e., target utterance 12) that was intended for the device 110 so that the output 182 generated by the backend speech system 180 is not degraded by the background interference.

[0034] In the example shown, the backend speech system 180 includes an ASR system 190 that employs an ASR model 192 to process the enhanced input speech features 262 to generate a speech recognition result (e.g., transcription) for the target utterance 12. The ASR system 190 may further include a natural language understanding (NLU) module (not shown) that performs semantic interpretation on the transcription of the target utterance 12 to identify the query/command directed toward the device 110. As such, the output 182 from the backend speech system 180 may include the transcription and/or instructions to fulfill the query/command identified by the NLU module.

[0035] The backend speech system 180 may additionally or alternatively include a hotword detection model (not shown) configured to detect whether or not the enhanced input speech features 262 include a presence of one or more hotwords/warm words the hotword detection model is trained to detect. For instance, the hotword detection model may output a hotword detection score indicating a likelihood that the enhanced input speech features 262 corresponding to the target utterance 12 include a particular hotword/warm word. Detection of a hotword may trigger a wake-up process that causes the device 110 to wake-up from a sleep state. For instance, the device 110 may wake-up and process the hotword and/or one or more terms preceding/following the hotword.

[0036] In additional examples, the background speech system 180 includes an audio or audio-video calling application (e.g., a video conferencing application). Here, the enhanced input speech features 262 corresponding to the target utterance 12 are used by the audio or audio-video calling application to filter the voice of the target speaker 10

for communications to recipients during an audio or audio-video communication session. The background speech system 180 may additionally or alternatively include a speaker identification model configured to perform speaker identification using the enhanced input speech features 262 to identify the user 10 that spoke the target utterance 12.

[0037] In the example shown, the device 110 captures the multichannel noisy input signal 202 (also referred to as audio data) of the target utterance 12 spoken by the user 10 in the presence of background interference emanating from one or more sources other than the user 10. The multichannel noisy input signal 202 includes one or more single channel noisy input signals 206, 206a-n of audio. The device 110 may correspond to any computing device associated with the user 10 and capable of receiving multichannel noisy input signals 202. Some examples of user devices 110 include, but are not limited to, mobile devices (e.g., mobile phones, tablets, laptops, etc.), computers, wearable devices (e.g., smart watches, smart headphones, etc.), smart appliances, and internet of things (IoT) devices, smart speakers, etc. The device 110 includes data processing hardware 112 and memory hardware 114 in communication with the data processing hardware 112 and storing instructions, that when executed by the data processing hardware 112, cause the data processing hardware 112 to perform one or more operations. The multichannel neural frontend speech enhancement model 200 may execute on the data processing hardware 112. In some examples, the backend speech system 180 executes on the data processing hardware 112. In some examples, the device 110 includes one or more applications (i.e., software applications) where each application may utilize enhanced input speech features 262 generated by the multichannel neural frontend speech enhancement model 200 to perform various functions within the application. For instance, the device 110 includes an assistant application configured to communicate synthesized playback audio 154 to the user 10 to assist the user 10 with various tasks.

[0038] The user device 110 further includes (or is in communication with) an audio subsystem with an array of audio capturing devices (e.g., microphones) 116, 116a-n for capturing and converting spoken utterances 12 within the speech environment into electrical signals and a speech output device (e.g., a speaker) 118 for communicating an audible audio signal (e.g., a synthesized playback audio 154 from the device 110). Each microphone 116 in the array of microphones 116 of the user device 110 may separately record the utterance 12 on a separate dedicated channel 206 of the multichannel noisy input signal 202. For example, the user device 110 may include two microphones 116 that each record the utterance 12, and the recordings from the two microphones 116 may be combined into a two-channel noisy input signal 202 (i.e., stereophonic audio or stereo). That is, the two microphones reside on the user device 110. In some examples, the user device 110 includes more than two microphones 116. Additionally or alternatively, the user device 102 may be in communication with two or more microphones 116 separate/remote from the user device 110. For example, the user device 110 may be a mobile device disposed within a vehicle and in wired or wireless communication (e.g., Bluetooth) with two or more microphones 116 of the vehicle. In some configurations, the user device 110 is in communication with least one microphone 116 residing on a separate device 111, which may include, without

limitation, an in-vehicle audio system, a computing device, a speaker, or another user device. In these configurations, the separate device 111 may also be in communication with the one or more microphones 116 residing on the user device 110.

[0039] In some examples, the device 110 is configured to communicate with a remote system 130 via a network (not shown). The remote system 130 may include remote resources 132, such as remote data processing hardware 134 (e.g., remote servers or CPUs) and/or remote memory hardware 136 (e.g., remote databases or other storage hardware). The device 110 may utilize the remote resources 132 to perform various functionality related to speech processing and/or synthesized playback communication. The multichannel neural frontend speech enhancement model 200 and the backend speech system 180 may reside on the device 110 (referred to as on-device systems) or reside remotely (e.g., reside on the remote system 130), but in communication with the device 110. In some examples, one or more backend speech systems 180 reside locally or on-device while one or more other backend speech systems 180 reside remotely. In other words, one or more backend speech systems 180 leveraging the enhanced input speech features 262 output from the multichannel neural frontend speech enhancement model 200 may be local or remote in any combination. For instance, when a system 180 is rather large in size or processing requirements, the system 180 may reside in the remote system 130. Yet when the device 110 may support the size or the processing requirements of one or more systems 180, the one or more systems 180 may reside on the device 110 using the data processing hardware 112 and/or the memory hardware 114. Optionally, the one or more of the systems 180 may reside on both locally/on-device and remotely. For instance, a backend speech system 180 may default to execute on the remote system 130 when a connection between the device 110 and remote system 130 is available, but when the connection is lost or unavailable, the system 180 instead executes locally on the device 110.

[0040] In some implementations, the device 110 or a system associated with the device 110 identifies text that the device 110 will communicate to the user 10 as a response to a query spoken by the user 10. The device 110 may then use a text-to-speech (TTS) system to convert the text into corresponding synthesized playback audio 154 for the device 110 to communicate to the user 10 (e.g., audibly communicate to the user 10) as the response to the query. Once generated, the TTS system communicates the synthesized playback audio 154 to the device 110 to allow the device 110 to output the synthesized playback audio 154. For instance, the device 110 outputs the synthesized playback audio 154 of “today is sunny” at a speaker 118 of the device 110 responsive to the user 10 providing a spoken query for today’s weather forecast.

[0041] With continued reference to FIG. 1, when the device 110 outputs the synthesized playback audio 154, the synthesized playback audio 154 generates an echo 156 captured by the audio capturing device 116. The synthesized playback audio 154 corresponds to a reference audio signal. While synthesized playback audio 154 depicts a reference audio signal in the example of FIG. 1, the reference audio signal may include other types of playback audio 154 including media content output from the speaker 118 or a communication from a remote user the user 10 is conversing with (e.g., voice over IP call or video conferencing call)

through the device 110. Unfortunately, in addition to the echo 156, the audio capturing device 116 may also be simultaneously capturing the target utterance 12 spoken by the user 10 that includes a follow-up query inquiring more about the weather, by stating “what about tomorrow?” For example, FIG. 1 depicts that, as the device 110 outputs the synthesized playback audio 154, the user 10 inquires more about the weather, in a spoken utterance 12 to the device 110, by stating “what about tomorrow?” Here, the spoken utterance 12 and the echo 156 are both captured at the audio capturing device 116 simultaneously to form the multichannel noisy input signal 202. In other words, the multichannel noisy input signal 202 includes an overlapped audio signal where some portion of the target utterance 12 spoken by the user 10 overlaps with some portion of the reference audio signal (e.g., synthesized playback audio) 154 output from the speaker 118 of the device 110. In addition to the synthesized playback audio 154, competing speech 13 spoken by another user 11 in the environment, as well as non-speech characteristics such as a ringtone 15 from a separate user device 111 may also be captured by the audio capturing device 116 and contribute to background interference that overlaps with the target utterance 12.

[0042] In FIG. 1, the backend speech system 180 may have issues processing the target utterance 12 corresponding to the follow-up weather query “what about tomorrow?” in the multichannel noisy input signal 202 due to the presence of the background interference attributed to at least one of the playback audio 154, competing speech 13, or non-speech background noise 15 interfering with target utterance 12. The multichannel neural frontend speech enhancement model 200 is employed to improve robustness of the backend speech system 180 by effectively removing (i.e., masking) the presence of the background interference recorded by the device 110 when the user 10 spoke the target utterance 12.

[0043] The model 200 may perform speech enhancement by applying noise context modeling where the speech cleaner 300 of the model 200 processes the multichannel contextual noise signal 204 associated with a predetermined duration of noise segments captured by the audio capturing device 116 prior to the target utterance 12 spoken by the user 10. In some examples, the predetermined duration includes six (6) seconds of noise segments. As such, the multichannel contextual noise signal 204 provides noise context. In some examples, the multichannel contextual noise signal 204 includes linear blind frequency-domain equalization (LFBE) features of the noise context signal for use as contextual information.

[0044] FIG. 2 shows the multichannel neural frontend speech enhancement model 200 of FIG. 1. The multichannel neural frontend speech enhancement model 200 uses a modified version of a conformer neural network architecture that combines convolution and self-attention to model short-range and long-range interactions. The model 200 includes a speech cleaner 300, short-time Fourier transform (STFT) modules 210, 210a-b, a first pre-processing block 220, 220a, a second pre-processing block 220, 220b, a feature stack 230, an encoder 240, a first masking layer 250, 250a, a second masking layer 250, 250b, and a phase corrector 260. The speech cleaner 300 may execute an adaptive noise cancellation algorithm (FIG. 3). The pre-processing blocks 220a, 220b may execute streaming pre-processing that utilizes a combination of sub- and full-band layers to aggregate

both spectral and temporal information (FIG. 4). The encoder 240 may include a stack of self-attention blocks 500 (FIG. 5).

[0045] FIG. 3 provides an example adaptive noise cancellation algorithm executed by the speech cleaner 300. Here, the speech cleaner 300 includes an FIR module 310 including an FIR filter, a minimization module 320, and a cancellation module 330. The speech cleaner 300 may be configured to receive, as input, the multichannel noisy input signal 202 and the multichannel contextual noise signal 204 and generate, as output, a single channel cleaned input signal 340. Here, the speech cleaner 300 includes a finite impulse response (FIR) filter to process the multichannel noisy input signal 202. The speech cleaner 300 may operate on an STFT transformed signal with a fast Fourier transform (FFT) size 2048, a window size of 128 milliseconds (ms) and a shift of 64 ms. Here, the temporal context may be $D^{CK}=3$ with a forgetting factor of $\alpha^{CL}=0.995$.

[0046] In the example shown, for simplicity, the multichannel noisy input signal 202 includes three channels 206a-c each including respective audio features captured by a separate dedicated microphone 116a-c in an array of three microphones 116. However, as mentioned above, the front-end speech enhancement model 200 is agnostic to a number of microphones 116 in the array of microphones 116. In other words, the multichannel noisy input signal 202 can include one channel 206 captured by one microphone 116, two channels 206 captured by two microphones 116, or four or more channels 206 captured by four or more microphones 116 without departing from the scope of the present disclosure.

[0047] Here, the FIR module 310 applies the FIR filter on all channels 206 of the multichannel noisy input signal 202 except for a first channel 206a to generate a summed output 312. In other words, the FIR module 310 does not process the first channel 206a of the multichannel noisy input signal 202 but does apply the FIR filter on the second channel 206b and the third channel 206c of the multichannel noisy input signal 202 to generate the summed output 312. The minimization module 320 receives the summed output 312 and the first channel 206a and generates a minimized output 322 by subtracting the summed output 312 from the first channel 206a of the multichannel noisy input signal 202. Mathematically, the FIR filter includes a tapped delay line of length L of three (3) applied to the channels 206b, 206c but not the channel 206a, where determining the minimized output 322 may be expressed as follows:

$$Z_m(n) = Y_0(n) - \sum_{l=-}^{L-1} U_m^H \tilde{Y}_m(k, n-1) \quad (1)$$

[0048] where $\tilde{Y}_m$ is a vector of time delayed Short-time Fourier transform (STFT) processed input for the channels 206b, 206c and $U_m(k)$ is a vector of the filter coefficients to be applied to the channels 206b, 206c. I_m and $U_m(k)$ may be expressed as follows:

$$\tilde{Y}_m(n) = [Y_m(n), Y_m(n-1), \dots, Y_m(n-(L-1))]^T \quad (2)$$

$$U_m(k) = [U_m(k, 0), U_m(k, 1), \dots, U_m(k, N-1)]^T \quad (3)$$

[0049] where the filter coefficients may minimize the power of the output as follows:

$$\hat{U}_m = \arg \min_{U_m} E_n[|Z_m(n)|^2] \quad (4)$$

[0050] Because the speech cleaner 300 is implemented on the device 110, the cancellation module 330 may use the multichannel contextual noise signal 204 that occurs directly before the utterance 12 in the multichannel noisy input signal 202. In other words, the minimization module 320 generates the minimized output 322 through adaptation during the multichannel contextual noise signal 204 when the utterance 12 is not present in the multichannel noisy input signal 202 (i.e., non-target speech frames). The adaptation may include a recursive least squares (RLS) algorithm. Once the speech cleaner 300 detects the utterance 12, the filter coefficients are frozen, where the cancellation module 330 applies the last coefficients before the utterance 12 to the multichannel noisy input signal 202 to cancel the background interference to produce the single channel cleaned input signal 340 as follows:

$$\hat{X}(n) = Y_0(n) = \sum_{l=-}^{L-1} \hat{U}_m^H \hat{X}_m(k, n-1) \quad (5)$$

[0051] Referring back to FIG. 2, the STFT modules 210a, 210b are each configured to convert the respective audio features of the single channel cleaned input signal 340 and a single channel 206a of the multichannel noisy input signal 202 into the STFT spectra, whereby the converted respective audio features of the single channel cleaned input signal 340 and a single channel 206a of the multichannel noisy input signal 202 are provided as STFT coefficients 212. Each STFT module 210 may use an FFT size of 512, a window size of 32 ms and a shift of 16 ms. As shown, a first STFT module 210, 210a receives, as input, the single channel cleaned input signal 340 and generates, as output, STFT coefficients 212a of the single channel cleaned input signal 340. Similarly, a second STFT module 210b receives, as input, the single channel 206a of the multichannel noisy input signal 202 and generates, as output, STFT coefficients 212b of the single channel noisy input signal 206a.

[0052] The first pre-processing block 220, 220a is configured to receive, as input, the STFT coefficients 212a of the single channel cleaned input signal 340 and generate as output a maximum value 454a of an embedding dimension of the single channel cleaned input signal 340. The pre-processing block 220a may include a plurality of pre-processing layers 400, where an initial pre-processing layer 400 of the plurality of pre-processing layers 400 receives the STFT coefficients 212a of the single channel cleaned input signal 340 output from the first STFT module 210a, and a final pre-processing layer 400 of the plurality of pre-processing layers 400 generates the maximum value 454a of the embedding dimension of the single channel cleaned input signal 340.

[0053] Similarly, the second pre-processing block 220, 220b is configured to receive, as input, the STFT coefficients 212b for the single channel noisy input signal 206a and generate, as output, a maximum value 454b of an embedding dimension of the single channel noisy input signal 206a.

Here, the second pre-processing block **220b** may include a plurality of pre-processing layers **400**, where an initial pre-processing layer **400** of the plurality of pre-processing layers **400** receives the STFT coefficients **212b** of the single channel noisy input signal **206a** output from the second STFT module **210b**, and a final pre-processing layer **400** of the plurality of pre-processing layers **400** generates the maximum value **454b** of the embedding dimension of the single channel noisy input signal **206a**.

[0054] FIG. 4 provides an example of a pre-processing layer **400** from the plurality of pre-processing layers **400** of the respective pre-processing blocks **220a**, **220b**. The pre-processing layer **400** includes a convolutional layer **410**, an unfold frequency (also referred to as F-Unfold) operation **420**, a first layer normalization **430**, **430a**, a non-causal context-aware layer **440**, a first deconvolutional layer **450**, **450a**, a residual connection **455**, an unfold time (also referred to as T-unfold) operation **460**, a second layer normalization **430**, **430b**, a causal context-aware layer **470**, and a second deconvolutional layer **450**, **450b**. To simplify the multichannel neural frontend speech enhancement model **200**, each pre-processing layer **400** only forwards the maximum value **454** rather than forwarding a full embedding vector **412**. The unfold window for each of the unfold frequency operation **420** and the unfold time operation **460** may use of $\tilde{F}=\tilde{T}=4$, where an embedding vector **412** has a size $E=64$. The non-causal context-aware layer **440** and the causal context-aware layer **470** may include a long-short term memory (LSTM)-based architecture where the non-causal context-aware layer **440** includes a bidirectional LSTM and the causal context-aware layer **470** includes a unidirectional LSTM, where each of the context-aware layers **440**, **470** includes one hidden layer with 64 nodes. In other implementations, the non-causal context-aware layer **440** and the causal context-aware layer **470** include a conformer-based architecture, where each of the context-aware layers **440**, **470** include a single Conformer block with eight (8) attention heads and a kernel size of 15×1 . In these implementations, the non-causal context-aware layer **440** uses a context window of the 30 prior and 30 future frequency bins, while the causal context-aware layer **470** uses the 30 prior frames as its context window.

[0055] With particular reference to each pre-processing layer **400** of the first pre-processor block **220a**, the convolutional layer **410** is configured to receive the maximum value 454_{j-1} dimension of each corresponding frame of the single channel cleaned input signal **340** output by a preceding pre-processor layer **400** and calculate a corresponding embedding vector **412** of size E for each of multiple time-frequency bins. In some instances, the convolutional layer **410** includes a causal two-dimensional convolutional layer with a 3×3 kernel and a stride equal to one (1). In these instances, the causal two-dimensional convolutional layer may be used to calculate the embedding vector of size E for each time-frequency bin. The unfold frequency operation **420** receives the corresponding embedding **412** and unfolds the corresponding embedding vector **412** over a frequency dimension f to generate an output **422**. For example, for each frequency bin the $\tilde{F}$ neighboring frequencies bins are stacked in the embedding dimension, resulting in an embedding dimension having a size $\tilde{F} \cdot E$.

[0056] The first layer normalization **430a** receives the output **422** generated by the unfold frequency operation **420** and normalizes each embedding vector **412** of the output

422 unfolded over the frequency dimension f to generate the output **432**. Subsequently, the non-causal context-aware layer **440** receives the output **432** generated by the first layer normalization **430a** and correlates the frequency dimension f as a context dimension to generate an output **442**. The output **422** of the non-causal context-aware layer **440** is next processed by the first deconvolutional layer **450** and combined with the corresponding embedding vectors **412** calculated by the convolutional layer **410** to generate updated embedding vectors **412U**. Thereafter, the updated embedding vectors **412U** are provided to the residual connection **455**.

[0057] The unfold time operation **460** receives the updated embedding vectors **412U** and unfolds the updated embedding vectors **412U** over a time dimension t to generate an output **462**. Here, for each frame $\tilde{T}$, past time frames may be stacked in the embedding dimension. The second layer normalization **430b** receives the output **462** generated by the unfold time operation **460** and normalizes each of the updated embedding vectors **412U** unfolded over the time dimension t to generate an output **434**. Subsequently, the causal context-aware layer **470** receives the output **434** generated by the second layer normalization **430b** and correlates the time dimension t as the context dimension to generate the output **472**. The second deconvolutional layer **450b** receives the output **472** of the causal context-aware layer **470** and the updated embedding vectors **412U** provided to the residual connection **455** and combines the output **472** and the updated embedding vectors **412U** to output the maximum value **454a** of the embedding dimension of the single channel cleaned input signal **340**.

[0058] With reference to each pre-processing layer **400** of the second pre-processor block **220b**, the convolutional layer **410** is configured to receive the maximum value 454_{j-1} dimension of each corresponding frame of the single channel noisy input signal **206a** output by a preceding pre-processor layer **400** and calculate a corresponding embedding vector **412** of size E for each of multiple time-frequency bins. In some instances, the convolutional layer **410** includes a causal two-dimensional convolutional layer with a 3×3 kernel and a stride equal to one (1). In these instances, the causal two-dimensional convolutional layer may be used to calculate the embedding vector of size E for each time-frequency bin. The unfold frequency operation **420** receives the corresponding embedding **412** and unfolds the corresponding embedding vector **412** over a frequency dimension f to generate an output **422**. For example, for each frequency bin the $\tilde{F}$ neighboring frequencies bins are stacked in the embedding dimension, resulting in an embedding dimension having a size $\tilde{F} \cdot E$.

[0059] The first layer normalization **430a** receives the output **422** generated by the unfold frequency operation **420** and normalizes each embedding vector **412** of the output **422** unfolded over the frequency dimension f to generate the output **432**. Subsequently, the non-causal context-aware layer **440** receives the output **432** generated by the first layer normalization **430a** and correlates the frequency dimension f as a context dimension to generate an output **442**. The output **422** of the non-causal context-aware layer **440** is next processed by the first deconvolutional layer **450** and combined with the corresponding embedding vectors **412** calculated by the convolutional layer **410** to generate updated

embedding vectors **412U**. Thereafter, the updated embedding vectors **412U** are provided to the residual connection **455**.

[0060] The unfold time operation **460** receives the updated embedding vectors **412U** and unfolds the updated embedding vectors **412U** over a time dimension t to generate an output **462**. Here, for each frame T , past time frames may be stacked in the embedding dimension. The second layer normalization **430b** receives the output **462** generated by the unfold time operation **460** and normalizes each of the updated embedding vectors **412U** unfolded over the time dimension t to generate an output **434**. Subsequently, the causal context-aware layer **470** receives the output **434** generated by the second layer normalization **430b** and correlates the time dimension t as the context dimension to generate the output **472**. The second deconvolutional layer **450b** receives the output **472** of the causal context-aware layer **470** and the updated embedding vectors **412U** provided to the residual connection **455** and combines the output **472** and the updated embedding vectors **412U** to output the maximum value **454b** of the embedding dimension of the single channel noisy output signal **206a**.

[0061] Referring back to FIG. 2, the feature stack **230** is configured to receive, as input, the maximum value **454a** of the embedding dimension of the single channel cleaned input signal **340** and the maximum value **454b** of the single channel noisy input signal **206a**, and generate a stacked input **232** including the maximum value **454a** of the embedding dimension of the single channel cleaned input signal **340** and the maximum value **454b** of the single channel noisy input signal **206a**. In some instances, the feature stack **230** concatenates the maximum value **454a** of the embedding dimension of the single channel cleaned input signal **340** and the maximum value **454b** of the single channel noisy input signal **206a** to generate the stacked input **232**.

[0062] The encoder **240** receives the stacked input **232** including the maximum value **454a** of the embedding dimension of the single channel cleaned input signal **340** and the maximum value **454b** of the single channel noisy input signal **206a**, and generates, as output, an un-masked output **580**. The encoder **240** includes a stack of self-attention blocks **500** (also referred to as blocks **500**). Here, an initial block **500** of the stack of self-attention blocks **500** receives the stacked input **232** including maximum value **454a** of the embedding dimension of the single channel cleaned input signal **340** and the maximum value **454b** of the single channel noisy input signal **206a** output from the pre-processing blocks **220a**, **220b**, and a final block **500** of the stack of self-attention blocks **500** generates the un-masked output **580**.

[0063] Each block **500** may include a feed-forward layer, a self-attention layer, a convolutional layer, and a second feed-forward layer. In some implementations, the stack of self-attention blocks **500** includes a stack of Conformer blocks **500**. In these implementations, the stack of Conformer blocks **500** includes four (4) layers of Conformer blocks **500** each with 1024 units, 8 attention heads, 15x1 convolutional kernel size, and 64 frames of self-attention to enable a streaming model. An example Conformer block **500** is described in greater detail below with reference to FIG. 5.

[0064] The first masking layer **250a** is configured to receive, as input, the un-masked output **580** output by the self-attention blocks **500** of the encoder **240**, and the STFT

coefficients **212a** of the single channel cleaned input signal **340** and generate, as output, a masked single channel cleaned input signal **340M**. To take advantage of the non-distorted parts of the single channel cleaned input signal **340**, the second masking layer **250b** is configured to receive, as input, the STFT coefficients **212b** for the single channel noisy input signal **206a** and the un-masked output **580** generated by the self-attention blocks **500** of the encoder **240**, and generate, as output, a masked single channel noisy input signal **206M**.

[0065] The phase extraction layer **260** receives, as input, the un-masked output **580**, the masked single channel cleaned input signal **340M**, and the masked single channel noisy input signal **206M** and generates, as output, enhanced input speech features **262** corresponding to the target utterance **12**. The phase extraction layer **260** may calculate a phase correction term $\hat{\Delta}\phi \in [-\pi, \pi]$ by processing the masked single channel cleaned input signal **340M**, and the masked single channel noisy input signal **206M** using a feed-forward layer with a tanh activation and multiplying the result with π . Mathematically, the phase extraction layer **260** may generate the enhanced input speech features **262**, as follows:

$$\hat{X}_{l,f} = \left(Y_{c_o,l,f} \cdot \hat{M}_{l,f}^Y + \hat{X}_{l,f}^{CL} \cdot \hat{M}_{l,f}^{CL} \right) \cdot e^{j\Delta\phi_{l,f}} \quad (6)$$

[0066] where $Y_{l,f}$ denotes the vector of the frequency representations of the signal at time frame l and frequency bin f , for all channels c of the multichannel noisy input signal **202**. $\hat{M}_{l,f}^Y$ denotes the masked single channel noisy input signal **206M**, while $\hat{M}_{l,f}^{CL}$ denotes the masked single channel cleaned input signal **340M**, and $\hat{X}_{l,f}^{CL}$ denotes the multi-microphone STFT coefficients **212** of reverberated target speech.

[0067] In other implementations, the model **200** includes a decoder (not shown) configured to decode the un-masked output **580**, the masked single channel cleaned input signal **340M**, and the masked single channel noisy input signal **206M** into the enhanced input speech features **262** corresponding to the target utterance **12**. Here, the decoder may include a simple projection decoder having a single layer, frame-wise fully connected network with sigmoid activation.

[0068] FIG. 5 provides an example of a block **500** from the stack of self-attention blocks **500** of the encoder **240**. The block **500** includes a first half feed-forward layer **510**, a second half feed-forward layer **440**, with a multi-head self-attention block **420** and a convolutional layer **430** disposed between the first and second half feed-forward layers **510**, **440**, and concatenation operators **505**, **505a-d**. The first half feed-forward layer **510** processes the stacked input **232** including the maximum value **454a** of the embedding dimension of the single channel cleaned input signal **340** and the maximum value **454b** of the single channel noisy input signal **206a** and generates an output **512**. Next, a first concatenation operator **505a** concatenates the output **512** with the stacked input **232** to generate a first concatenated input **514**. Subsequently, the multi-head self-attention block **420** receives the first concatenated input **514** and generates a noise summary **522**. Intuitively, the role of the multi-head self-attention block **420** is to summarize noise context separately for each input frame that is to be enhanced.

[0069] Next, a second concatenation operator **505b** concatenates the output noise summary **522** with the first concatenated input **514** to generate a second concatenated input **524**. Subsequently, the convolutional layer **530** subsamples the second concatenated input **524** including the noise summary **522** of the multi-head self-attention block **520** and the first concatenated input **514** and generates a convolutional output **532**. Thereafter, a third concatenation operator **505c** concatenates the convolutional output **532** with the second concatenated input **524** to generate a third concatenated input **534**. The third concatenated input **534** is provided as input to the second half-feed forward layer **540**, which generates an output **542**. The output **542** of the second half-feed forward layer **540** is concatenated with the third concatenated input **534** by a fourth concatenation operator **505d** to generate a fourth concatenated input **544**. Finally, the layernorm module **550** processes the fourth concatenated input **544** from the second half feed-forward layer **540**. Mathematically, the block **500** transforms input features x , using modulation features m , to produce output features y , as follows:

$$\begin{aligned}
 \hat{x} &= x + r(m) \odot x + h(m) \\
 \hat{x} &= \hat{x} + \frac{1}{2}FFN(\hat{x}), \hat{n} = n + \frac{1}{2}FFN(n) \\
 x' &= \hat{x} + Conv(\hat{x}), n' = \hat{n} + Conv(\hat{n}) \\
 x'' &= x' + MHCA(x', n') \\
 x''' &= x' \odot r(x'') + h(x'') \\
 x'''' &= x' + MHCA(x', x''') \\
 y &= LayerNorm\left(x'''' + \frac{1}{2}FFN(x''')\right)
 \end{aligned} \tag{7}$$

[0070] The block **500** generates, as an output, the unmasked output **580**, which is passed on to the next layer of the self-attention blocks **500**. Thus, the inputs **454a**, **454b** are modulated by each of the self-attention blocks **500**.

[0071] FIG. 6 shows an example training process **600** for computing ASR loss **560** when the frontend speech enhancement model **200** is trained jointly with the ASR model **192**. The training process **600** may execute on the remote system **130** of FIG. 1. As shown, the training process **600** obtains one or more training data sets **620** stored in a data store **610** and trains the multichannel neural frontend speech enhancement model **200** on the training data sets **620**. The data store **610** may reside on the memory hardware **136** of the remote system **130**. Each training data set **620** includes a plurality of training examples, **630**, **630a-n**, where each training example **630** may include a training utterance **632**. Here, only an encoder **640** of the ASR model **192** is used for computing the loss. The ASR loss **660** is computed as the **12** distance between the outputs of the ASR encoder **640** for target features **636** of the training utterance **632** and the enhanced input speech features **262**. The ASR encoder **640** is not updated during the training process **600**. In detail, the training process **600** computes the ASR loss **660** by generating, using the ASR encoder **640** of the ASR model **192** configured to receive the enhanced input speech features **262** predicted by the frontend speech enhancement model **200** for a training utterance **632** as input, predicted outputs **622** of the ASR encoder **640** for the enhanced input speech

features **262**, and generating, using the ASR encoder **640** configured to receive target speech features **636** for the training utterance **632** as input, target outputs **624** of the ASR encoder **640** for the target speech features **636**. The predicted outputs **622** for the enhanced input speech features **262** and the target outputs **624** for the target speech features **636** may each include respective sequences of LFBE features. Thereafter, the training process **600**, via a loss module **650**, computes the ASR loss **660** based on the predicted outputs **622** of the ASR encoder **640** for the enhanced input speech features **262** and the target outputs **624** of the ASR encoder **640** for the target speech features **636**. The goal of using the ASR loss **660** is to make enhancements to the frontend speech enhancement model **200** to be more attuned to the ASR model **192**, which is critical for getting the best performance out of the frontend speech enhancement model **200**. Notably, the ASR loss **660** is applied to the frontend speech enhancement model **200** in the time domain signal. By keeping the parameters of the ASR model **192** fixed, the ASR model **192** is decoupled from the frontend speech enhancement model **200**, thereby allowing each to be trained and deployed independent of each other.

[0072] In some implementations, the frontend speech enhancement model **200** is trained jointly with the ASR model **192** of the backend automatic speech recognition system **180** using a spectral loss (e.g., scale invariant signal-to-noise ratio (SI-SNR) loss) and the ASR loss **660**. In these implementations, the training target **636** for training the multichannel neural frontend speech enhancement model **200** uses an ideal ratio mask (IRM). IRMs may be computed using reverberant speech and reverberant noise based on an assumption that speech and noise are uncorrelated in Mel spectral space as follows:

$$M(t, f) = \frac{X(t, f)}{X(t, f) + N(t, f)} \tag{8}$$

Here, X and N are the reverberant speech and reverberant noise Mel spectrograms, respectively. t and f represent time and Mel frequency bin indices. The choice to estimate IRMs is based on the targets being bounded between $[0, 1]$, simplifying the estimation process. Moreover, the ASR model **192** used for evaluation may be trained on real and simulated reverberant data, resulting in a trained ASR model **192** that is relatively robust to reverberant speech. Therefore, IRMs derived using reverberant speech as the target still provide substantial gains in performance. The spectral loss during training may be computed based on L1 and L2 losses between the IRM and estimated IRM, $\hat{M}$ as follows:

$$\mathcal{L} = \sum_{t,f} |M(t, f) - \hat{M}(t, f)| + (M(t, f) - \hat{M}(t, f))^2 \tag{9}$$

$$\text{Where } L1 = |M(t, f) - \hat{M}(t, f)|, \text{ and } L2 = (M(t, f) - \hat{M}(t, f))^2$$

[0073] During inference, the estimated IRM is scaled and floored to reduce speech distortion at the expense of reduced noise suppression. This is especially important, since the ASR model **192** is sensitive to speech distortions and non-linear frontend processing, which is one of the main

challenges in improving performance of robust ASR models using enhancement frontends. The enhanced feature may be derived as follows:

$$\hat{X}(t, f) = Y(t, f) \odot \max(\hat{M}(t, f), \beta)^\alpha \quad (10)$$

Here, Y is the noisy Mel spectrogram, $\hat{X}$ is an estimate of clean Mel spectrogram, α and β are exponential mask scalars, and the mask floor. In some examples, α is set to 0.5, and β is set to 0.01. The enhanced features may be log-compressed, i.e. $\log(\hat{X})$, and passed to the ASR model 192 for evaluation.

[0074] FIG. 7 includes a flowchart of an example arrangement of operations for a method 700 of performing automatic speech recognition using a multichannel neural frontend speech enhancement model 200. The method 700 may execute on data processing hardware 810 (FIG. 8) (e.g., data processing hardware 112 of the user device 110 or remote data processing hardware 134 of the remote system 130) based on instructions stored on memory hardware 820 (FIG. 8) (e.g., memory hardware 114 of the user device 110 or remote memory hardware 136 of the remote system 130). At operation 702, the method 700 includes receiving short-time Fourier transform (STFT) coefficients 212a of a single channel cleaned input signal 340 and STFT coefficients 212b for a single channel noisy input signal 206a. Here, the single channel cleaned input signal 340 is generated from a multichannel noisy input signal 206, 206a-n and a multichannel contextual noise signal 204.

[0075] At operation 704, the method 700 includes generating, using a first pre-processor block 220, 220a of a speech enhancement model 200, a maximum value 454a of an embedding dimension of the single channel cleaned input signal 340. The method 700 also includes, at operation 706, generating, using a second pre-processor block 220, 220b of the speech enhancement model 200, a maximum value 454b of an embedding dimension of the single channel noisy input signal 206a. At operation 708, the method 700 further includes generating, as output from a stack of self-attention blocks 500 of the speech enhancement model 200 configured to receive a stacked input 232 including the maximum value 454a of the embedding dimension of the single channel cleaned input signal 340 output from the first pre-processor block 400a and the maximum value 454b of the embedding dimension of the single channel noisy input signal 206a output from the second pre-processor block 400b, an un-masked output 580. Here, each self-attention block 500 in the stack of self-attention blocks 500 includes a multi-headed attention mechanism.

[0076] The method 700 also includes, at operation 710, generating, using a first masking layer 250, 250a of the speech enhancement model 200 configured to receive the STFT coefficients 212a for the single channel cleaned input signal 340 and the un-masked output 580 generated as output from the final block 500 of the stack of self-attention blocks 500, a masked single channel cleaned input signal 340M. At operation 712, the method 700 further includes generating, using a second masking layer 250, 250b of the speech enhancement model 200 configured to receive the STFT coefficients 212b for the single channel noisy input signal 206a and the un-masked output 580 generated as output from the final block 500 of the stack of self-attention

blocks 500, a masked single channel noisy input signal 206M. The method 700 also includes, at operation 714, generating, using a phrase extraction layer 260 of the speech enhancement model 200 configured to receive the un-masked output 580, the masked single channel cleaned input signal 340M, and the masked single channel noisy input signal 206M, enhanced input speech features 262 corresponding to a target utterance 12.

[0077] FIG. 8 is schematic view of an example computing device 800 that may be used to implement the systems and methods described in this document. The computing device 800 is intended to represent various forms of digital computers, such as laptops, desktops, workstations, personal digital assistants, servers, blade servers, mainframes, and other appropriate computers. The components shown here, their connections and relationships, and their functions, are meant to be exemplary only, and are not meant to limit implementations of the disclosures described and/or claimed in this document.

[0078] The computing device 800 includes a processor 810, memory 820, a storage device 830, a high-speed interface/controller 840 connecting to the memory 820 and high-speed expansion ports 850, and a low-speed interface/controller 860 connecting to a low-speed bus 870 and a storage device 830. Each of the components 810, 820, 830, 840, 850, and 860, are interconnected using various busses, and may be mounted on a common motherboard or in other manners as appropriate. The processor 810 (e.g., data processing hardware 112, 134 of FIG. 1) can process instructions for execution within the computing device 800, including instructions stored in the memory 820 or on the storage device 830 to display graphical information for a graphical user interface (GUI) on an external input/output device, such as display 880 coupled to high-speed interface 840. In other implementations, multiple processors and/or multiple buses may be used, as appropriate, along with multiple memories and types of memory. Also, multiple computing devices 800 may be connected, with each device providing portions of the necessary operations (e.g., as a server bank, a group of blade servers, or a multi-processor system).

[0079] The memory 820 (e.g., memory hardware 114, 136 of FIG. 1) stores information non-transitorily within the computing device 800. The memory 820 may be a computer-readable medium, a volatile memory unit(s), or non-volatile memory unit(s). The non-transitory memory 820 may be physical devices used to store programs (e.g., sequences of instructions) or data (e.g., program state information) on a temporary or permanent basis for use by the computing device 800. Examples of non-volatile memory include, but are not limited to, flash memory and read-only memory (ROM)/programmable read-only memory (PROM)/erasable programmable read-only memory (EPROM)/electronically erasable programmable read-only memory (EEPROM) (e.g., typically used for firmware, such as boot programs). Examples of volatile memory include, but are not limited to, random access memory (RAM), dynamic random access memory (DRAM), static random access memory (SRAM), phase change memory (PCM) as well as disks or tapes.

[0080] The storage device 830 is capable of providing mass storage for the computing device 800. In some implementations, the storage device 830 is a computer-readable medium. In various different implementations, the storage device 830 may be a floppy disk device, a hard disk device, an optical disk device, or a tape device, a flash memory or

other similar solid state memory device, or an array of devices, including devices in a storage area network or other configurations. In additional implementations, a computer program product is tangibly embodied in an information carrier. The computer program product contains instructions that, when executed, perform one or more methods, such as those described above. The information carrier is a computer- or machine-readable medium, such as the memory **820**, the storage device **830**, or memory on processor **810**.

[0081] The high-speed controller **840** manages bandwidth-intensive operations for the computing device **800**, while the low-speed controller **860** manages lower bandwidth-intensive operations. Such allocation of duties is exemplary only. In some implementations, the high-speed controller **840** is coupled to the memory **820**, the display **880** (e.g., through a graphics processor or accelerator), and to the high-speed expansion ports **850**, which may accept various expansion cards (not shown). In some implementations, the low-speed controller **860** is coupled to the storage device **830** and a low-speed expansion port **890**. The low-speed expansion port **890**, which may include various communication ports (e.g., USB, Bluetooth, Ethernet, wireless Ethernet), may be coupled to one or more input/output devices, such as a keyboard, a pointing device, a scanner, or a networking device such as a switch or router, e.g., through a network adapter.

[0082] The computing device **800** may be implemented in a number of different forms, as shown in the figure. For example, it may be implemented as a standard server **800a** or multiple times in a group of such servers **800a**, as a laptop computer **800b**, or as part of a rack server system **800c**.

[0083] Various implementations of the systems and techniques described herein can be realized in digital electronic and/or optical circuitry, integrated circuitry, specially designed ASICs (application specific integrated circuits), computer hardware, firmware, software, and/or combinations thereof. These various implementations can include implementation in one or more computer programs that are executable and/or interpretable on a programmable system including at least one programmable processor, which may be special or general purpose, coupled to receive data and instructions from, and to transmit data and instructions to, a storage system, at least one input device, and at least one output device.

[0084] A software application (i.e., a software resource) may refer to computer software that causes a computing device to perform a task. In some examples, a software application may be referred to as an “application,” an “app,” or a “program.” Example applications include, but are not limited to, system diagnostic applications, system management applications, system maintenance applications, word processing applications, spreadsheet applications, messaging applications, media streaming applications, social networking applications, and gaming applications.

[0085] The non-transitory memory may be physical devices used to store programs (e.g., sequences of instructions) or data (e.g., program state information) on a temporary or permanent basis for use by a computing device. The non-transitory memory may be volatile and/or non-volatile addressable semiconductor memory. Examples of non-volatile memory include, but are not limited to, flash memory and read-only memory (ROM)/programmable read-only memory (PROM)/erasable programmable read-only memory (EPROM)/electronically erasable programmable

read-only memory (EEPROM) (e.g., typically used for firmware, such as boot programs). Examples of volatile memory include, but are not limited to, random access memory (RAM), dynamic random access memory (DRAM), static random access memory (SRAM), phase change memory (PCM) as well as disks or tapes.

[0086] These computer programs (also known as programs, software, software applications or code) include machine instructions for a programmable processor, and can be implemented in a high-level procedural and/or object-oriented programming language, and/or in assembly/machine language. As used herein, the terms “machine-readable medium” and “computer-readable medium” refer to any computer program product, non-transitory computer readable medium, apparatus and/or device (e.g., magnetic discs, optical disks, memory, Programmable Logic Devices (PLDs)) used to provide machine instructions and/or data to a programmable processor, including a machine-readable medium that receives machine instructions as a machine-readable signal. The term “machine-readable signal” refers to any signal used to provide machine instructions and/or data to a programmable processor.

[0087] The processes and logic flows described in this specification can be performed by one or more programmable processors, also referred to as data processing hardware, executing one or more computer programs to perform functions by operating on input data and generating output. The processes and logic flows can also be performed by special purpose logic circuitry, e.g., an FPGA (field programmable gate array) or an ASIC (application specific integrated circuit). Processors suitable for the execution of a computer program include, by way of example, both general and special purpose microprocessors, and any one or more processors of any kind of digital computer. Generally, a processor will receive instructions and data from a read only memory or a random access memory or both. The essential elements of a computer are a processor for performing instructions and one or more memory devices for storing instructions and data. Generally, a computer will also include, or be operatively coupled to receive data from or transfer data to, or both, one or more mass storage devices for storing data, e.g., magnetic, magneto optical disks, or optical disks. However, a computer need not have such devices. Computer readable media suitable for storing computer program instructions and data include all forms of non-volatile memory, media and memory devices, including by way of example semiconductor memory devices, e.g., EPROM, EEPROM, and flash memory devices; magnetic disks, e.g., internal hard disks or removable disks; magneto optical disks; and CD ROM and DVD-ROM disks. The processor and the memory can be supplemented by, or incorporated in, special purpose logic circuitry.

[0088] To provide for interaction with a user, one or more aspects of the disclosure can be implemented on a computer having a display device, e.g., a CRT (cathode ray tube), LCD (liquid crystal display) monitor, or touch screen for displaying information to the user and optionally a keyboard and a pointing device, e.g., a mouse or a trackball, by which the user can provide input to the computer. Other kinds of devices can be used to provide interaction with a user as well; for example, feedback provided to the user can be any form of sensory feedback, e.g., visual feedback, auditory feedback, or tactile feedback; and input from the user can be received in any form, including acoustic, speech, or tactile

input. In addition, a computer can interact with a user by sending documents to and receiving documents from a device that is used by the user; for example, by sending web pages to a web browser on a user's client device in response to requests received from the web browser.

[0089] A number of implementations have been described. Nevertheless, it will be understood that various modifications may be made without departing from the spirit and scope of the disclosure. Accordingly, other implementations are within the scope of the following claims.

What is claimed is:

1. A multichannel neural frontend speech enhancement model for speech recognition, the speech enhancement model comprising:

- a first pre-processor block configured to:
 - receive, as input, short-time Fourier transform (STFT) coefficients for a single channel cleaned input signal, the single channel cleaned input signal generated from a multichannel noisy input signal and a multichannel contextual noise signal; and
 - generate, as output, a maximum value of an embedding dimension of the single channel cleaned input signal;
- a second pre-processor block configured to:
 - receive, as input, STFT coefficients for a single channel noisy input signal; and
 - generate, as output, a maximum value of an embedding dimension of the single channel noisy input signal;
- a stack of self-attention blocks each having a multi-headed self attention mechanism, the stack of self-attention blocks configured to:
 - receive, as input, at an initial block of the stack of self-attention blocks, a stacked input comprising the maximum value of the embedding dimension of the single channel cleaned input signal output from the first pre-processor block and the maximum value of the embedding dimension of the single channel noisy input signal output from the second pre-processor block; and
 - generate, as output, from a final block of the stack of self-attention blocks, an un-masked output;
- a first masking layer configured to:
 - receive, as input, the STFT coefficients for the single channel cleaned input signal and the un-masked output generated as output from the final block of the stack of self-attention blocks; and
 - generate, as output, a masked single channel cleaned input signal;
- a second masking layer configured to:
 - receive, as input, the STFT coefficients for the single channel noisy input signal and the un-masked output generated as output from the final block of the stack of self-attention blocks; and
 - generate, as output, a masked single channel noisy input signal; and
- a phrase extraction layer configured to:
 - receive, as input, the un-masked output, the masked single channel cleaned input signal, and the masked single channel noisy input signal; and
 - generate, as output, enhanced input speech features corresponding to a target utterance.

2. The speech enhancement model of claim 1, wherein the first pre-processor block comprises a plurality of pre-processing layers, each pre-processing layer comprising:

- a convolutional layer configured to receive the maximum value of the embedding dimension of each corresponding frame of the single channel cleaned input signal output by a preceding pre-processing layer and calculate a corresponding embedding vector of size E for each of multiple time-frequency bins;
- an unfold frequency operation configured to unfold the corresponding embedding vector over a frequency dimension;
- a first layer normalization configured to normalize each embedding vector;
- a non-causal context-aware layer configured to correlate the frequency dimension as a context dimension;
- a first deconvolutional layer configured to process and combine an output of the non-causal context-aware layer with the corresponding embedding vectors calculated by the convolutional layer to provide updated embedding vectors to a residual connection;
- an unfold time operation configured to unfold the updated embedding vectors over a time dimension;
- a second layer normalization configured to normalize each of the updated embedding vectors unfolded over the time dimension;
- a causal context-aware layer configured to correlate the time dimension as the context dimension; and
- a second deconvolutional layer configured to process and combine an output of the causal context-aware layer with the updated embedding vectors provided to the residual connection to output the maximum value of the embedding dimension of the single channel cleaned input signal.

3. The speech enhancement model of claim 2, wherein the convolutional layer comprises a causal two-dimensional convolutional layer with a 3×3 kernel and a stride equal to one.

4. The speech enhancement model of claim 1, wherein the second pre-processor block comprises a plurality of pre-processing layers, each pre-processing layer comprising:

- a convolutional layer configured to receive the maximum value of the embedding dimension of each corresponding frame of the single channel noisy input signal output by a preceding pre-processing layer and calculate a corresponding embedding vector of size E for each of multiple time-frequency bins;
- an unfold frequency operation configured to unfold the corresponding embedding vector over a frequency dimension;
- a first layer normalization configured to normalize each embedding vector;
- a non-causal context-aware layer configured to correlate the frequency dimension as a context dimension;
- a first deconvolutional layer configured to process and combine an output of the non-causal context-aware layer with the corresponding embedding vectors calculated by the convolutional layer to provide updated embedding vectors to a residual connection;
- an unfold time operation configured to unfold the updated embedding vectors over a time dimension;
- a second layer normalization configured to normalize each of the updated embedding vectors unfolded over the time dimension;
- a causal context-aware layer configured to correlate the time dimension as the context dimension; and

- a second deconvolutional layer configured to process and combine an output of the causal context-aware layer with the updated embedding vectors provided to the residual connection to output the maximum value of the embedding dimension of single channel noisy input signal.
5. The speech enhancement model of claim 4, wherein the convolutional layer comprises a causal two-dimensional convolutional layer with a 3×3 kernel and a stride equal to one.
6. The speech enhancement model of claim 1, wherein the stack of self-attention blocks comprises a stack of Conformer blocks.
7. The speech enhancement model of claim 6, wherein the stack of Conformer blocks comprises four Conformer blocks.
8. The speech enhancement model of claim 1, wherein the speech enhancement model executes on data processing hardware residing on a user device, the user device configured to capture the target utterance and the multichannel contextual noise signal via an array of microphones of the user device.
9. The speech enhancement model of claim 1, wherein the speech enhancement model is agnostic to a number of microphones in the array of microphones.
10. The speech enhancement model of claim 1, further comprising a speech cleaner executing an adaptive noise cancelation algorithm that generates the single channel cleaned input signal by:
- applying a finite impulse response (FIR) filter on all channels of the multichannel noisy input signal except for a first channel of the multichannel noisy input signal to generate a summed output; and
 - subtracting the summed output from the first channel of the multichannel noisy input signal.
11. The speech enhancement model of claim 1, wherein a backend speech system is configured to process the enhanced input speech features corresponding to the target utterance.
12. The speech enhancement model of claim 11, wherein the backend speech system comprises at least one of an automatic speech recognition (ASR) model or an audio or audio-video calling application.
13. The speech enhancement model of claim 1, wherein the speech enhancement model is trained jointly with a backend automatic speech recognition (ASR) model using a spectral loss and an ASR loss.
14. The speech enhancement model of claim 13, wherein the spectral loss is based on an L1 loss function and L2 loss function distance between an estimated ratio mask and an ideal ratio mask, the ideal ratio mask computed using reverberant speech and reverberant noise.
15. The speech enhancement model of claim 13, wherein the ASR loss is computed by:
- generating, using an ASR encoder of the ASR model configured to receive enhanced speech features predicted by the speech enhancement model for a training utterance as input, predicted outputs of the ASR encoder for the enhanced speech features;
 - generating, using the ASR encoder configured to receive target speech features for the training utterance as input, target outputs of the ASR encoder for the target speech features; and
 - computing the ASR loss based on the predicted outputs of the ASR encoder for the enhanced speech features and the target outputs of the ASR encoder for the target speech features.
16. A computer-implemented method that when executed on data processing hardware causes the data processing hardware to perform operations comprising:
- receiving short-time Fourier transform (STFT) coefficients for a single channel cleaned input signal and STFT coefficients for a single channel noisy input signal, the single channel cleaned input signal generated from a multichannel noisy input signal and a multichannel contextual noise signal;
 - generating, using a first pre-processor block of a speech enhancement model, a maximum value of an embedding dimension of the single channel cleaned input signal;
 - generating, using a second pre-processor block of the speech enhancement model, a maximum value of an embedding dimension of the single channel noisy input signal;
 - generating, as output from a stack of self-attention blocks of the speech enhancement model configured to receive a stacked input comprising the maximum value of the embedding dimension of the single channel cleaned input signal output from the first pre-processor block and the maximum value of the embedding dimension of the single channel noisy input signal output from the second pre-processor block, an un-masked output, wherein each self-attention block in the stack of self-attention blocks comprises a multi-headed self attention mechanism;
 - generating, using a first masking layer of the speech enhancement model configured to receive the STFT coefficients for the single channel cleaned input signal and the un-masked output generated as output from the final block of the stack of self-attention blocks, a masked single channel cleaned input signal;
 - generating, using a second masking layer of the speech enhancement model configured to receive the STFT coefficients for the single channel noisy input signal and the un-masked output generated as output from the final block of the stack of self-attention blocks, a masked single channel noisy input signal; and
 - generating, using a phrase extraction layer of the speech enhancement model configured to receive the un-masked output, the masked single channel cleaned input signal, and the masked single channel noisy input signal, enhanced input speech features corresponding to a target utterance.
17. The computer-implemented method of claim 16 wherein the first pre-processor block comprises a plurality of pre-processing layers, each pre-processing layer comprising:
- a convolutional layer configured to receive the maximum value of the embedding dimension of each corresponding frame of the single channel cleaned input signal output by a preceding pre-processing layer and calculate a corresponding embedding vector of size E for each of multiple time-frequency bins;
 - an unfold frequency operation configured to unfold the corresponding embedding vector over a frequency dimension;

a first layer normalization configured to normalize each embedding vector;

a non-causal context-aware layer configured to correlate the frequency dimension as a context dimension;

a first deconvolutional layer configured to process and combine an output of the non-causal context-aware layer with the corresponding embedding vectors calculated by the convolutional layer to provide updated embedding vectors to a residual connection;

an unfold time operation configured to unfold the updated embedding vectors over a time dimension;

a second layer normalization configured to normalize each of the updated embedding vectors unfolded over the time dimension;

a causal context-aware layer configured to correlate the time dimension as the context dimension; and

a second deconvolutional layer configured to process and combine an output of the causal context-aware layer with the updated embedding vectors provided to the residual connection to output the maximum value of the embedding dimension of the single channel cleaned input signal.

18. The computer-implemented method of claim **17**, wherein the convolutional layer comprises a causal two-dimensional convolutional layer with a 3×3 kernel and a stride equal to one.

19. The computer-implemented method of claim **16**, wherein the second pre-processor block comprises a plurality of pre-processing layers, each pre-processing layer comprising:

- a convolutional layer configured to receive the maximum value of the embedding dimension of each corresponding frame of the single channel noisy input signal output by a preceding pre-processing layer and calculate a corresponding embedding vector of size E for each of multiple time-frequency bins;
- an unfold frequency operation configured to unfold the corresponding embedding vector over a frequency dimension;
- a first layer normalization configured to normalize each embedding vector;
- a non-causal context-aware layer configured to correlate the frequency dimension as a context dimension;
- a first deconvolutional layer configured to process and combine an output of the non-causal context-aware layer with the corresponding embedding vectors calculated by the convolutional layer to provide updated embedding vectors to a residual connection;
- an unfold time operation configured to unfold the updated embedding vectors over a time dimension;
- a second layer normalization configured to normalize each of the updated embedding vectors unfolded over the time dimension;
- a causal context-aware layer configured to correlate the time dimension as the context dimension; and
- a second deconvolutional layer configured to process and combine an output of the causal context-aware layer with the updated embedding vectors provided to the residual connection to output the maximum value of the embedding dimension of single channel noisy input signal.

20. The computer-implemented method of claim **19**, wherein the convolutional layer comprises a causal two-dimensional convolutional layer with a 3×3 kernel and a stride equal to one.

21. The computer-implemented method of claim **16**, wherein the stack of self-attention blocks comprises a stack of Conformer blocks.

22. The computer-implemented method of claim **21**, wherein the stack of Conformer blocks comprises four Conformer blocks.

23. The computer-implemented method of claim **16**, wherein the speech enhancement model executes on data processing hardware residing on a user device, the user device configured to capture the target utterance and the multichannel contextual noise signal via an array of microphones of the user device.

24. The computer-implemented method of claim **16**, wherein the speech enhancement model is agnostic to a number of microphones in the array of microphones.

25. The computer-implemented method of claim **16**, wherein speech enhancement model further comprises a speech cleaner executing an adaptive noise cancellation algorithm that generates the single channel cleaned input signal by:

- applying a finite impulse response (FIR) filter on all channels of the multichannel noisy input signal except for a first channel of the multichannel noisy input signal to generate a summed output; and

- subtracting the summed output from the first channel of the multichannel noisy input signal.

26. The computer-implemented method of claim **16**, wherein a backend speech system is configured to process the enhanced input speech features corresponding to the target utterance.

27. The computer-implemented method of claim **26**, wherein the backend speech system comprises at least one of an automatic speech recognition (ASR) model or an audio or audio-video calling application.

28. The computer-implemented method of claim **16**, wherein the speech enhancement model is trained jointly with a backend automatic speech recognition (ASR) model using a spectral loss and an ASR loss.

29. The computer-implemented method of claim **28**, wherein the spectral loss is based on an L1 loss function and L2 loss function distance between an estimated ratio mask and an ideal ratio mask, the ideal ratio mask computed using reverberant speech and reverberant noise.

30. The computer-implemented method of claim **28**, wherein the ASR loss is computed by:

- generating, using an ASR encoder of the ASR model configured to receive enhanced speech features predicted by the speech enhancement model for a training utterance as input, predicted outputs of the ASR encoder for the enhanced speech features;

- generating, using the ASR encoder configured to receive target speech features for the training utterance as input, target outputs of the ASR encoder for the target speech features; and

- computing the ASR loss based on the predicted outputs of the ASR encoder for the enhanced speech features and the target outputs of the ASR encoder for the target speech features.

* * * * *